

SISTEME DE ASISTENȚĂ RUTIERĂ

Candidat: Dragoș Dumitru Roșu

Coordonator științific: Asist. dr. fiz. ing. Septimiu Lica

Sesiunea: Iunie 2025

CUPRINS

1	INTRODUCERE	5
1.1	INFORMAȚII GENERALE	5
1.2	SCOPUL LUCRĂRII	6
2	GENERALITĂȚI	7
2.1	RASPBERRY PI	7
2.2	PiCAMERA	9
2.3	MODULUL DE AFIȘARE	10
2.4	MODULUL POWERBANK	10
2.5	PYTHON.....	11
2.6	OPENCV.....	11
3	SISTEM DE DETECȚIE A BENZILOR DE CIRCULAȚIE	13
3.1	CORECTAREA DISTORSIUNILOR.....	13
3.2	IMPLEMENTAREA DETECȚIEI BENZILOR DE CIRCULAȚIE	17
4	SISTEM DE DETECȚIE A INDICATOARELOR RUTIERE.....	26
4.1	INSTALAREA SISTEMULUI DE OPERARE PE RASPBERRY PI	26
4.2	CONFIGURAREA MEDIULUI DE DEZVOLTARE	28
4.2.1	Instalare OpenCV	28
4.2.2	Instalare TensorFlow Lite Runtime	28
4.2.3	Instalare Picamera2.....	28
4.2.4	Instalarea suportului pentru afișajul LCD1602	29
4.3	Modelul de recunoaștere a indicatoarelor rutiere	29
4.4	IMPLEMENTAREA DETECȚIEI INDICATOARELOR RUTIERE	31
4.5	REZULTATE EXPERIMENTALE	37
5	CONCLUZII	42
	BIBLIOGRAFIE	43

REZUMAT

Lucrarea de față urmărește dezvoltarea a două aplicații distincte, dar complementare, construite pe baza tehnologiilor de viziune artificială, cu scopul de a contribui la creșterea siguranței rutiere. Cele două sisteme abordează probleme diferite – unul este orientat către menținerea vehiculului în banda de circulație, iar celălalt urmărește înțelegerea corectă a semnalizării rutiere. Împreună, cele două soluții formează o fundație tehnică solidă și de viitor, pentru un sistem de asistență rutieră inteligentă.

Prima aplicație este un sistem teoretic de detecție a benzilor de circulație, realizat în limbajul **Python**, cu ajutorul bibliotecii **OpenCV**. Aceasta analizează un videoclip deja existent în computerul de test, filmat din perspectiva unui autovehicul, simulând funcționarea unui asistent de menținere pe bandă. Procesul include corectarea distorsiunii camerei pe baza unor parametri de calibrare, conversia imaginii în spațiul de culoare HLS pentru separarea canalelor relevante, aplicarea filtrului Sobel pentru detecția marginilor, binarizarea imaginii și transformarea perspectivei într-o vedere de sus (bird's eye view). Identificarea benzilor se realizează printr-un algoritm de tip sliding window. Rezultatul este suprapus peste imaginea originală. Deși, această aplicație nu a fost integrată hardware, ea oferă o schemă completă și funcțională de procesare, fiind o bază pentru implementări practice ulterioare.

Cea de-a doua aplicație este un sistem complet **practic și funcțional**, implementat tot în **Python**, pe platforma **Raspberry Pi**. Acesta, are rolul de a detecta în timp real indicatoarele rutiere folosind o cameră **Picamera2**. Imaginea este prelucrată prin detecția contururilor și a formelor geometrice (cerc, triunghi, pătrat), urmată de extragerea regiunii de interes (ROI). Aceasta este redimensionată și transmisă către un model de rețea neuronală, în format **TensorFlow Lite**, antrenat pentru clasificarea indicatoarelor rutiere. Rezultatul este afișat pe un ecran **LCD16x2**, iar sistemul include o logică de afișare optimizată pentru a evita repetarea inutilă a informațiilor. Întregul sistem funcționează autonom, acesta fiind alimentat de către un modul, **Powerbank 18650**, care integrează două celule **Li-Ion Samsung INR18650-25R**, asigurând astfel portabilitate și independență energetică în condiții reale.

Prin combinarea celor două componente – una axată pe menținerea vehiculului în bandă și cealaltă pe recunoașterea mediului rutier –, această lucrare oferă un exemplu practic de extindere a sistemelor de asistență rutieră. Sunt demonstrate atât capacitățile procesării clasice a imaginilor, cât și beneficiile folosirii rețelelor neuronale ușoare, adaptate platformelor embedded. Această abordare dublă scoate în evidență potențialul soluțiilor accesibile în construirea unor sisteme semi-autonome, cu aplicabilitate atât în domeniul cercetării, cât și în viitoarele dezvoltări comerciale.

ABSTRACT

This bachelor's thesis presents the development of two complementary applications based on artificial vision techniques, aiming to increase road safety through intelligent driving assistance. The first application is a theoretical lane detection system, implemented in **Python** using the **OpenCV** library. It processes a video shot from a vehicle's perspective, applying steps such as camera distortion correction, image conversion to HLS space, Sobel filtering, perspective transformation and lane identification using the sliding window algorithm. The result is superimposed on the original image, providing a complete simulation of a lane keeping assistant.

The second application is a complete hardware system, developed on the **Raspberry Pi** platform, that detects and classifies road signs in real time. The system uses a **Picamera2** camera, processes the image by identifying relevant geometric shapes and transmits the extracted regions to an optimized neural network model in **TensorFlow** Lite format. The classification results are displayed on an **LCD1602** screen, and autonomous operation is ensured by a **Powerbank** module with two Li-Ion cells, guaranteeing portability in real conditions.

By combining classical image processing with lightweight neural networks, the paper proposes a dual, practical and accessible approach, with potential for application in research or semi-autonomous commercial systems.

1 INTRODUCERE

În ultimii ani, progresele în inteligența artificială și procesarea imaginilor au condus, pe lângă altele, și la dezvoltarea unor soluții tot mai bune pentru automatizarea și siguranța condusului. Lucrarea de față explorează două direcții esențiale ale sistemelor inteligente de asistență rutieră: detectarea benzilor de circulație și recunoașterea indicatoarelor rutiere. Proiectul propune atât o abordare teoretică, cât și una practică, utilizând platforme accesibile și tehnici moderne de viziune artificială pentru implementarea acestor soluții . [1]

1.1 INFORMAȚII GENERALE

Automatizarea conducerii vehiculelor a devenit una dintre cele mai importante direcții de cercetare și inovare din domeniul transporturilor moderne. Primele încercări notabile au apărut încă din anii '80-'90, odată cu introducerea senzorilor digitali și a algoritmilor de procesare a imaginilor. Un moment decisiv l-a reprezentat competiția DARPA Grand Challenge din 2004, care a adus în prim-plan conceptul de vehicule capabile să navigheze autonom pe distanțe lungi [2]. Evoluțiile rapide din inteligența artificială, procesarea imaginilor și miniaturizarea hardware-ului au accelerat ulterior dezvoltarea sistemelor avansate de asistență pentru șoferi (ADAS) și a vehiculelor autonome [2].

Detectarea benzilor de circulație și recunoașterea indicatoarelor rutiere reprezintă două tehnologii-cheie în cadrul sistemelor ADAS, având un rol esențial în sporirea siguranței rutiere. Aceste soluții permit vehiculelor să se mențină corect în bandă, să respecte semnalizarea rutieră și să reacționeze eficient la schimbările din trafic. Totodată, ele sprijină șoferul prin furnizarea de informații critice în timp real, contribuind la reducerea riscului de accidente [3].

În acest context, Raspberry Pi s-a impus ca o soluție hardware populară, accesibilă și extrem de versatilă, utilizată atât în scopuri educaționale, cât și pentru realizarea de prototipuri industriale. Modelul Raspberry Pi 3, folosit în cadrul acestei lucrări, oferă resurse suficiente pentru procesarea imaginilor, în ciuda dimensiunilor compacte și a consumului energetic redus. Această platformă impune însă dezvoltarea unor soluții software atent optimizate, capabile să ruleze algoritmi de detecție vizuală în timp real [4] .

Biblioteca OpenCV este un instrument esențial în procesarea imaginilor, datorită capacității sale de a implementa algoritmi atât complecși, cât și eficienți. Cu suport extins pentru limbajul Python, OpenCV permite aplicarea de filtre, realizarea de transformări geometrice, conversii de culoare și analize morfologice, fiind folosită în această lucrare pentru detecția benzilor de circulație [5].

Pentru recunoașterea indicatoarelor rutiere, proiectul se bazează pe tehnici de clasificare utilizând modele de rețea neuronală optimizate în format TensorFlow Lite, adaptate pentru rulare eficientă pe platforma embedded. Camera Pi utilizată oferă captură de imagini în timp real, iar semnele sunt analizate în funcție de formă și culoare, apoi clasificate și afișate pe un ecran LCD1602 [6] .

Sistemele ADAS funcționează prin colectarea și procesarea continuă a informațiilor provenite de la senzori vizuali și alți senzori de proximitate. Datele obținute sunt analizate

În timp real pentru a genera decizii automate sau asistate, sprijinind astfel șoferul în diverse situații de trafic [4] [6]. Diagrama din Figura 2.1 ilustrează arhitectura generală a unui sistem ADAS, evidențiind componentele principale: senzorii, modulele de procesare și funcțiile de ieșire. În cadrul lucrării de față, sunt dezvoltate două funcționalități majore: detecția benzilor de circulație și recunoașterea semnelor rutiere, ambele ilustrate în schemă.

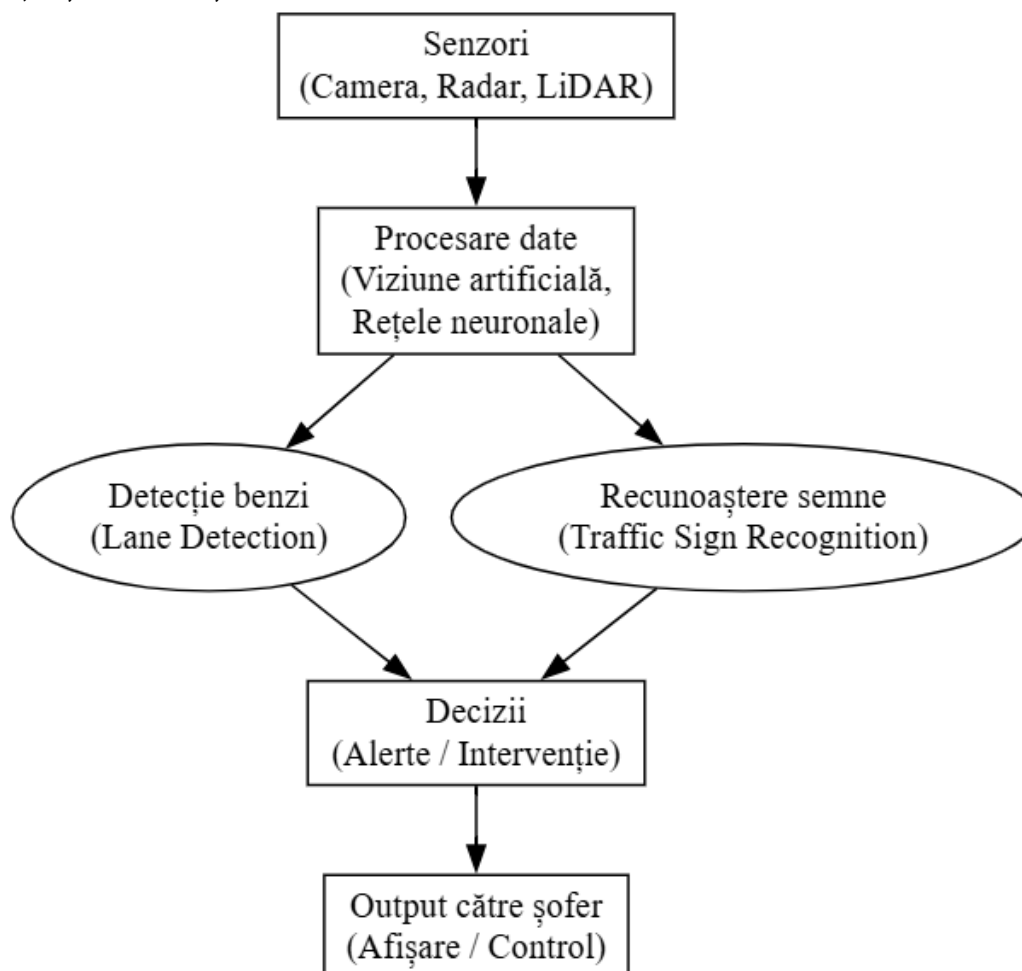


Figura 1 – Diagramă ADAS

1.2 SCOPUL LUCRĂRII

Scopul principal al acestei lucrări este proiectarea și implementarea a două aplicații complementare de viziune artificială, dedicate detecției benzilor de circulație și recunoașterii indicatoarelor rutiere, utilizând o platformă embedded accesibilă, Raspberry Pi 3. Ambele aplicații au fost concepute să funcționeze în timp real și să fie optimizate special pentru rularea pe sisteme cu resurse hardware limitate.

Detectarea benzilor de circulație este abordată în această lucrare dintr-o perspectivă teoretică, utilizând imagini video existente, unde algoritmi dezvoltați simulează funcționarea unui sistem de menținere pe bandă. În schimb, recunoașterea indicatoarelor rutiere este realizată într-un mod practic, prin captură live, clasificare automată și afișare a

rezultatelor pe un ecran, sistemul fiind alimentat portabil pentru a asigura autonomie completă.

Prin reunirea celor două direcții, lucrarea evidențiază potențialul tehnologiilor de viziune artificială în dezvoltarea de soluții practice pentru asistența rutieră inteligentă, maleabile, atât pentru scenarii de testare, cât și pentru implementare în condiții reale.

2 GENERALITĂȚI

2.1 RASPBERRY PI

Raspberry Pi este un computer de tip SBC (Single-Board Computer), având dimensiunile unui card de credit, produs în Regatul Unit de către Raspberry Pi Foundation. Scopul principal al acestuia a fost promovarea învățării noțiunilor fundamentale de informatică în școli și sprijinirea educației tehnice încă de la nivelul de bază [7].

Tabelul 1 – Specificații Raspberry Pi [8]

Procesor	Broadcom BCM2837B0, Cortex-A53 64-bit SoC @ 1.4GHz
Memorie	1G
Conectivitate	• 2.4 GHz and 5 GHz IEEE 802.11b/g/n/ac wireless LAN, Bluetooth 4.2, BLE • Gigabit Ethernet over USB 2.0 (maximum throughput 300Mbps) • 4 × USB 2.0 interface
Video și sunet	• 1 x full size HDMI • MIPI DSI display port • MIPI CSI camera port • 4 pole stereo output and composite video port
Multimedia	H.264, MPEG-4 decode (1080p30); H.264 encode (1080p30); OpenGL ES 1.1, 2.0 graphics
SD card suport	Micro SD format for loading operating system and data storage
Temperatura optimă	0-50°C

Deși are dimensiuni reduse, de doar 85 mm x 56 mm, Raspberry Pi funcționează ca un computer în toată regula, acesta fiind capabil să ruleze sisteme de operare precum Linux sau Windows și să suporte aplicații variate: de la jocuri și editoare de text, până la medii de programare, redare multimedia, teleconferințe și acces la internet [4].

Principalele diferențe față de un calculator personal sau un laptop convențional sunt dimensiunea compactă și o putere de calcul mai modestă. Raspberry Pi nu dispune de ecran

sau tastatură integrate, însă permite conectarea facilă a componentelor electronice externe, precum senzori, ecrane LCD, motoare sau relee. Flexibilitate lui, îl face o alegere ideală pentru proiecte embedded sau aplicații educaționale. [4]

Componentele principale ale plăcii Raspberry Pi 3B sunt prezentate în Figura 2.1.1, care ilustrează arhitectura generală și interfețele [4] .

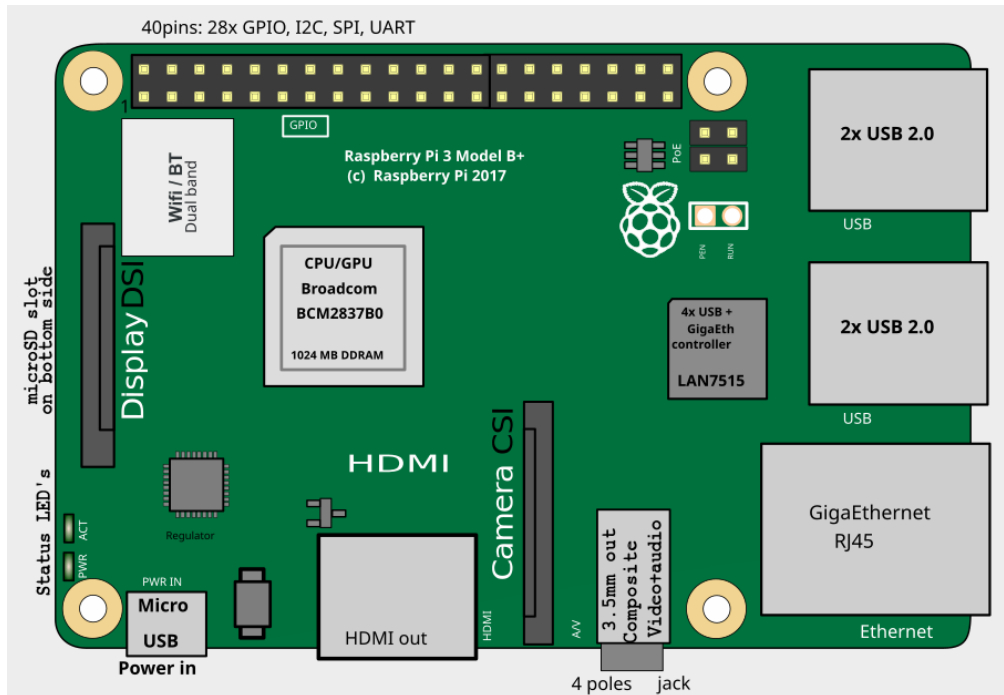


Figura 2.1.1 – Arhitectură Raspberry Pi

Procesor: Raspberry Pi utilizează sistemul Broadcom BCM2837B0 integrat pe cip, care include atât un procesor ARM, cât și o unitate de procesare grafică (GPU) cu nucleu video. Acesta este componenta centrală a plăcii, coordonând operațiunile tuturor dispozitivelor conectate și gestionând calculele necesare funcționării sistemului [4].

HDMI: Interfața multimedia (HDMI), este folosită pentru transmiterea semnalelor video și audio digitale către un monitor, cu un cablu compatibil [4].

Porturi GPIO: Raspberry Pi prezintă porturi de intrare/ieșire de uz general (GPIO), care permit interacțiunea cu diverse dispozitive externe, oferind o flexibilitate ridicată în realizarea proiectelor electronice [4].

Porturi USB: Raspberry Pi include porturi USB standard, prin care pot fi conectate periferice precum mouse-uri, tastaturi sau alte dispozitive de intrare/ieșire. De asemenea, acestea permit extinderea funcționalităților sistemului prin conectarea unor accesorii suplimentare, cum ar fi componente pentru a spori performanțele [4].

Card SD: Slotul pentru card este esențial pentru funcționarea plăcii, fiind necesar un card cu un sistem de operare preinstalat pentru pornirea și rularea dispozitivului [4].

Ethernet: Conectorul Ethernet, disponibil pe modelul Raspberry Pi 3B, oferă posibilitatea conectării rapide la rețele prin cablu [4].

Alimentare: Pentru alimentarea plăcii se utilizează un conector micro USB, compatibil cu surse de alimentare de 5V. [4].

Modulul camerei: Interfața CSI permite conectarea directă a camerei Pi la procesorul Broadcom, realizând astfel capturarea imaginilor [4].

2.2 PiCAMERA

Modulul Camera V2 poate fi utilizat atât pentru realizarea de materiale video în format Full HD, cât și pentru capturarea de fotografii statice, având în plus suport pentru modul HDR de până la 3 megapixeli. Funcționarea camerei este complet integrată în biblioteca *libcamera*. Beneficiază inclusiv de funcția rapidă de autofocus, ceea ce o face ușor de utilizat pentru începători, dar oferind în același timp multiple opțiuni de configurare pentru utilizatorii avansați. Camera este compatibilă cu toate modelele de Raspberry Pi [9].

Placa în sine este mică, de aproximativ 25mm x 20mm x 9mm, și cântărește puțin peste 3g, fiind ideală pentru aplicații mobile sau alte situații în care dimensiunile și greutatea sunt critice. Camera se conectează la Raspberry Pi printr-un cablu de panglică scurt după cum este ilustrat în Figura 2.2.1, folosind magistrala CSI — o interfață de mare viteză care transmite datele de pixel de la cameră către procesorul BCM2837B0. [9].

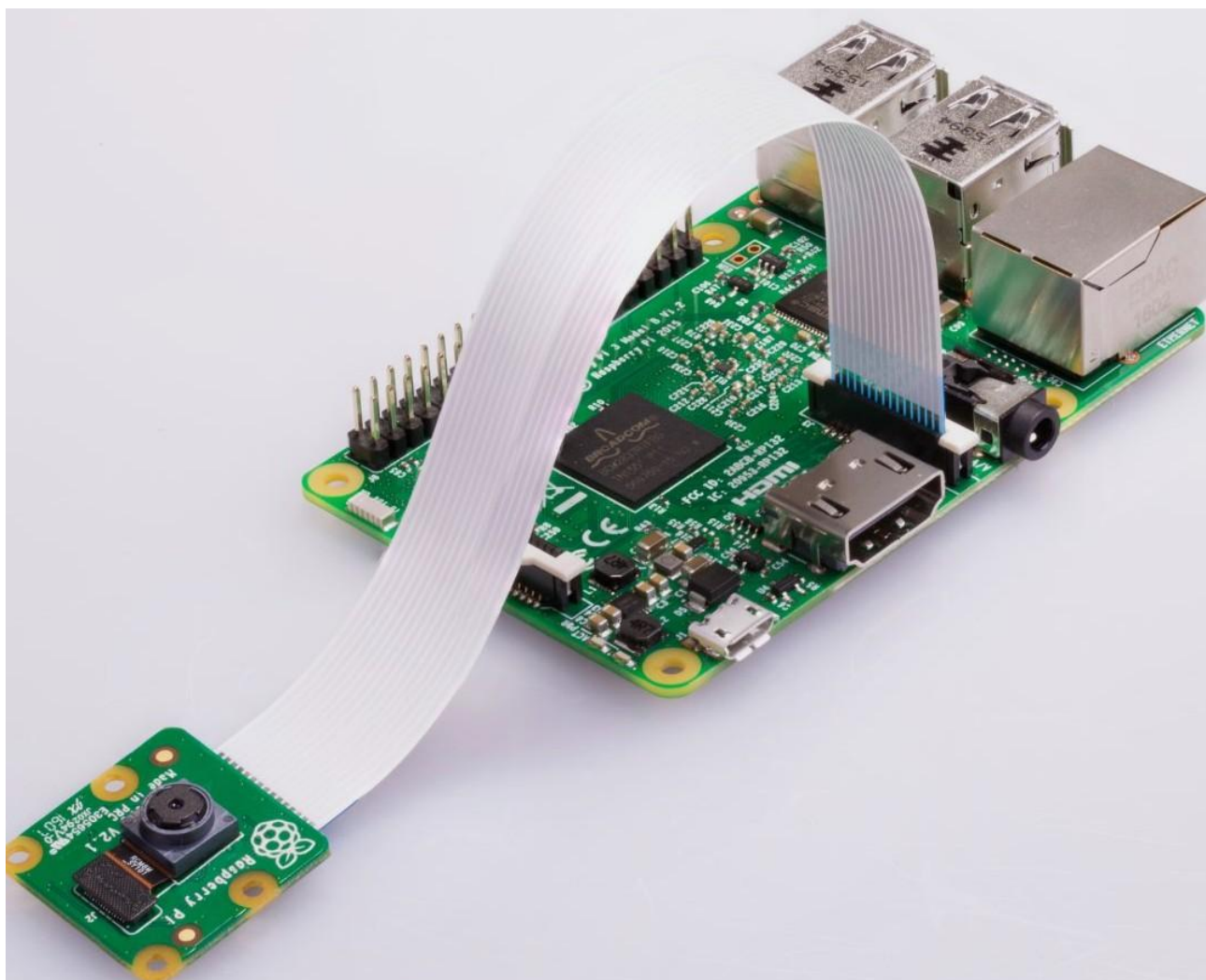


Figura 2.2.1 – Conectare PiCamera

2.3 MODULUL DE AFIȘARE

Modulul de afișare, LCD1602 creat de Waveshare, este un dispozitiv popular, utilizat frecvent pentru afișarea textului în aplicații bazate pe microcontrolere și în realizarea prototipurilor. Acesta dispune de un ecran cu 16 coloane și 2 rânduri, având iluminare de fundal albastră și text alb, ceea ce ușurează vizibilitatea informațiilor afișate. [10]

LCD-ul folosește interfața I2C, ceea ce îl face extrem de simplu de conectat la diverse microcontrolere, precum Arduino sau Raspberry Pi, necesitând un număr minim de pini de comunicație, așa cum este ilustrat în Figura 2.3.1 [10] .

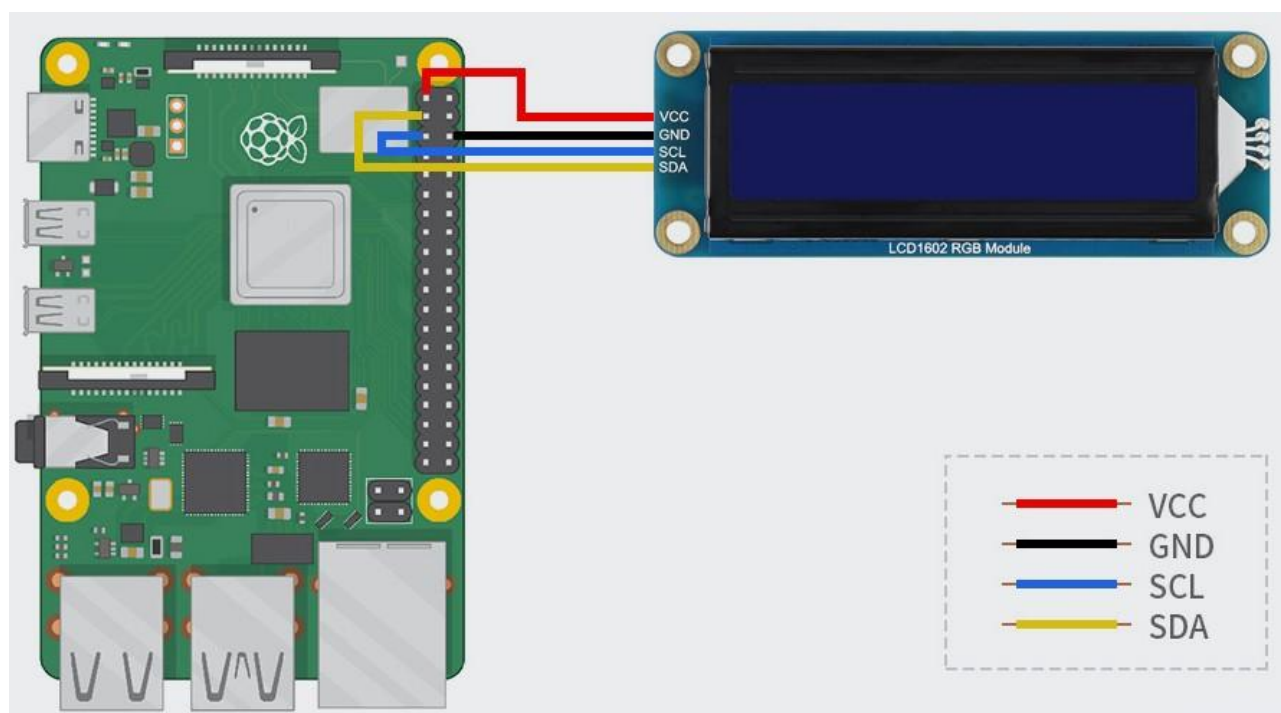


Figura 2.3.1 – Conectare LCD1602

2.4 MODULUL POWERBANK

Modulul powerbank 18650, configurat 2P, cu ieșire de 5V/3VDC și compatibil cu platforme precum Arduino și Raspberry Pi, este utilizat frecvent pentru alimentarea mobilă a circuitelor electronice și a proiectelor embedded. Acesta funcționează cu două baterii 18650 în paralel (2P), oferind o sursă de tensiune stabilă de 5V sau 3V, ideală pentru alimentarea dispozitivelor sensibile la variațiile de tensiune [11] .

Modulul este echipat cu porturi USB pentru ieșirea tensiunii, un port micro USB pentru încărcare, precum și cu circuite de protecție împotriva supratensiunii, supraîncărcării și scurtcircuitelor. Dimensiunile sale compacte și posibilitatea de integrare facilă îl fac o alegere excelentă pentru prototipuri mobile sau proiecte care necesită o sursă proprie de alimentare [11] .

2.5 PYTHON

Python este un limbaj de programare de nivel înalt, interpretat, caracterizat printr-o sintaxă clară și concisă. Creat în 1991 de Guido van Rossum, Python a câștigat rapid popularitate datorită ușurinței de utilizare, portabilității între diferite sisteme de operare și ecosistemului său bogat de biblioteci. Fiind un limbaj interpretat, Python elimină necesitatea compilării, ceea ce permite dezvoltarea rapidă și testarea imediată a codului – un avantaj esențial în aplicațiile prototip sau în cercetare [12].

Un alt punct forte, al acestuia, este suportul excelent pentru domenii precum inteligența artificială, procesarea imaginilor și automatizările, prin biblioteci consacrate precum OpenCV, NumPy sau TensorFlow. De asemenea, compatibilitatea cu platformele embedded, cum ar fi Raspberry Pi, îl transformă într-o alegere ideală pentru dezvoltarea de aplicații în timp real sau pentru sisteme cu resurse hardware limitate [13].

În ceea ce privește stilul de programare, Python impune utilizarea indentării codului pentru structurarea logică a programelor, renunțând astfel la acoladele întâlnite în limbaje precum C sau Java. Acest aspect favorizează scrierea unui cod curat, lizibil și ușor de întreținut, fiind o cerință importantă în proiectele moderne ce implică mentenanță sau colaborare între mai mulți dezvoltatori [13].

Astfel, Python nu este doar un limbaj ideal pentru începători, ci s-a impus ca un instrument solid și matur, utilizat pe scară largă în domenii critice precum automatizarea, robotica, analiza video sau sistemele de asistență rutieră [13].

2.6 OpenCV

OpenCV (Open Computer Vision) este o bibliotecă open-source dedicată procesării imaginilor și analizei video. Scrisă inițial în C/C++, aceasta este compatibilă cu sistemele de operare Windows, Linux și macOS și oferă interfețe pentru limbaje populare precum Python și Java, fiind utilizată pe scară largă în aplicațiile de viziune artificială [14].

Biblioteca pune la dispoziție funcții optimizate pentru prelucrarea imaginilor, detecția și segmentarea obiectelor, calibrarea camerelor, transformările de perspectivă, precum și urmărirea obiectelor în timp real. Datorită optimizării sale, multe dintre aceste operații pot fi implementate eficient chiar și pe sisteme cu resurse hardware limitate [14].

În OpenCV, imaginile sunt tratate sub forma unor matrici numerice, fiecare pixel conținând informații despre culoare sau luminozitate. De exemplu, într-o imagine alb-negru, fiecare pixel are o valoare între 0 (negru) și 255 (alb). Prin această metodă, imaginile devin matrici de valori numerice, așa cum este ilustrat în Figura 2.6.1, ceea ce permite aplicarea de filtre, detecția marginilor, evidențierea formelor geometrice sau urmărirea mișcărilor [14].

Dincolo de funcțiile sale clasice, OpenCV poate fi integrată cu TensorFlow, biblioteca de inteligență artificială dezvoltată de Google. Această combinație permite ca datele vizuale prelucrate să fie analizate de rețele neuronale pentru identificarea și clasificarea inteligentă a obiectelor. TensorFlow Lite, versiunea optimizată a bibliotecii, facilitează rularea acestor modele pe dispozitive embedded sau mobile, care au o putere de procesare limitată. [14].

Prin flexibilitatea și eficiența sa, OpenCV rămâne una dintre cele mai utilizate soluții pentru construcția sistemelor automate care „văd” și „înțeleg” mediul din jur pe baza imaginilor capturate [5].

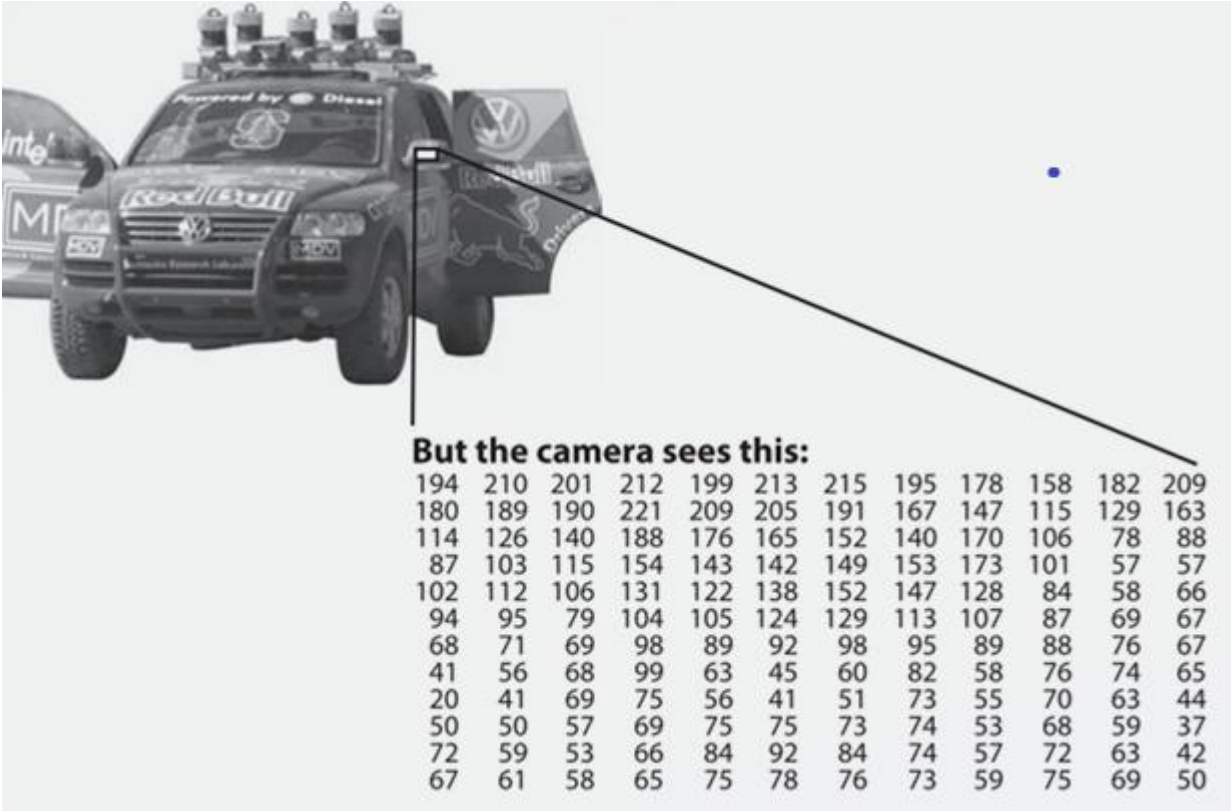


Figura 2.6.1 – Imaginea transformată în pixeli

3 SISTEM DE DETECȚIE A BENZILOR DE CIRCULAȚIE

Proiectul de față se concentrează pe dezvoltarea unui sistem de detecție a benzilor de circulație, utilizând tehnici moderne de viziune artificială. Scopul principal este realizarea unei aplicații capabile să analizeze un videoclip preexistent pentru a identifica benzile de circulație, contribuind astfel la îmbunătățirea soluțiilor de asistență rutieră..

Pentru realizarea acestui sistem, s-a utilizat biblioteca **OpenCV**, care permite procesarea eficientă a imaginii, inclusiv conversia la grayscale, aplicarea de filtre de estompare, detectarea marginilor cu algoritmi precum **Canny**, și transformarea perspectivei pentru obținerea unei vederi de sus (bird's-eye view). Algoritmul de **detecție a benzilor** folosește metoda **sliding window** pentru a identifica poziția acestora pe drum, iar rezultatele sunt afișate direct pe imaginea procesată.

Prin implementarea acestui sistem, proiectul propune să demonstreze cum tehnologiile de procesare a imaginilor pot fi utilizate pentru a spori siguranța rutieră. Astfel, oferind o soluție eficientă și portabilă de monitorizare a benzilor de circulație în timp real.

3.1 CORECTAREA DISTORSIUNILOR

Primul pas esențial în dezvoltarea unui sistem de detectare a benzilor de circulație constă în corectarea distorsiunilor introduse de lentilele camerei. Acestea distorsionează în mod inevitabil imaginea capturată, determinând ca liniile drepte să apară curbate și contururile să devină mai puțin clare. În Figura 3.1.1 este ilustrat modul în care distorsiunile afectează linii drepte: liniile care ar trebui să fie perfect paralele apar curbate, iar efectul devine mai accentuat pe măsură ce punctele se află mai departe de centrul imaginii. Acesta este un exemplu clasic de distorsiune radială, fenomen care determină deformarea liniilor drepte în curbe. Totodată, în imaginea, din partea dreaptă, este evidențiată și distorsiunea tangențială, rezultată dintr-un aliniament imperfect între lentilă și senzorul camerei. Ceea ce produce o deformare a imaginii în direcția tangențială față de centrul acesteia.



Figura 3.1.1 – Tipuri de distorsiuni

Pentru corectarea acestor distorsiuni, se utilizează procesul de calibrare a camerei, care implică folosirea unor imagini special pregătite pentru acest scop. În Figura 3.1.2 sunt prezentate exemple de imagini cu o tablă de șah, capturate din diferite unghiuri, utilizate în procesul de calibrare. Aceste imagini permit calcularea parametrilor interni ai camerei, precum matricea camerei și coeficienții de distorsiune, prin identificarea automată a colțurilor tablei de șah și stabilirea corespondenței lor cu pozițiile reale din spațiu.

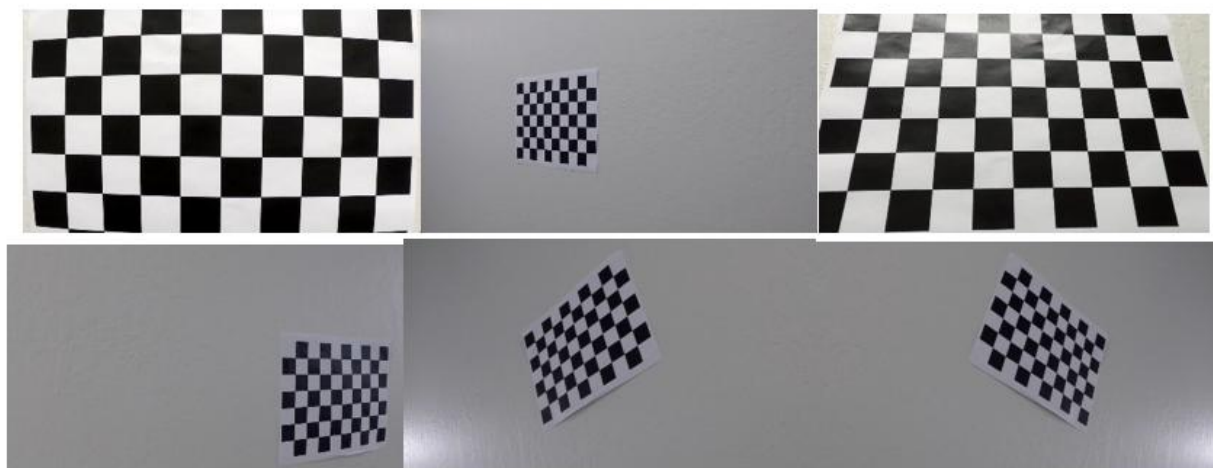


Figura 3.1.2 – Imaginile alea tablei de șah din diferite unghiuri

În Figura 3.1.3 este ilustrat sistemul de coordonate utilizat în procesul de calibrare a camerei. Coordonatele din lumea reală (sistemul de coordonate al lumii) sunt corelate cu sistemul de coordonate al camerei și, ulterior, cu coordonatele imaginii 2D capturate. Matricea intrinsecă descrie parametrii interni ai camerei, precum distanța focală și caracteristicile de distorsiune ale lentilei, în timp ce matricea extrinsecă definește relația dintre poziționarea obiectelor în spațiul real și poziția camerei. Aceste matrice sunt esențiale pentru corectarea distorsiunilor și pentru obținerea unei proiecții corecte a punctelor 3D din lumea reală într-o imagine 2D.

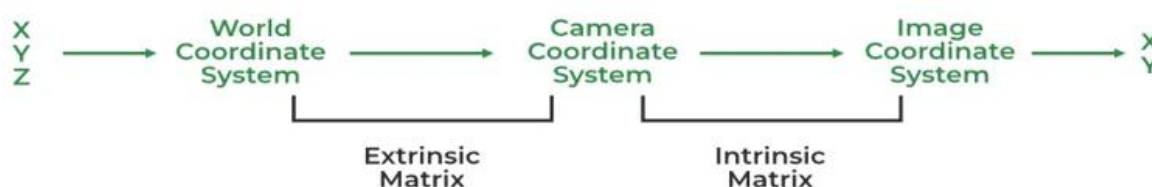


Figura 3.1.3 – Relația dintre lumea reală, cameră și imagine

În concluzie, procesul de calibrare a camerei, evidențiat în imaginile prezentate anterior, are un rol esențial în corectarea distorsiunilor optice introduse de lentile. Prin

calibrare, imaginea capturată reflectă realitatea într-un mod corect și precis, asigurând o bază fiabilă pentru procesările ulterioare. Această etapă este crucială pentru obținerea unor imagini acurate, care vor fi utilizate ulterior în analiza benzilor de circulație și în detecția indicatoarelor rutiere.

Pentru a corecta distorsiunile geometrice produse de lentila camerei și pentru a obține imagini conforme cu realitatea fizică, a fost aplicat un proces de calibrare utilizând biblioteca OpenCV. Procedura de calibrare se bazează pe un set de imagini care conțin o tablă de șah cu dimensiuni cunoscute (9x6 colțuri). Din aceste imagini sunt extrase punctele de control (colțurile tablei), care sunt ulterior corelate cu coordonatele 3D reale, definite într-un plan fix, așa cum este ilustrat în Figura 3.1.4.

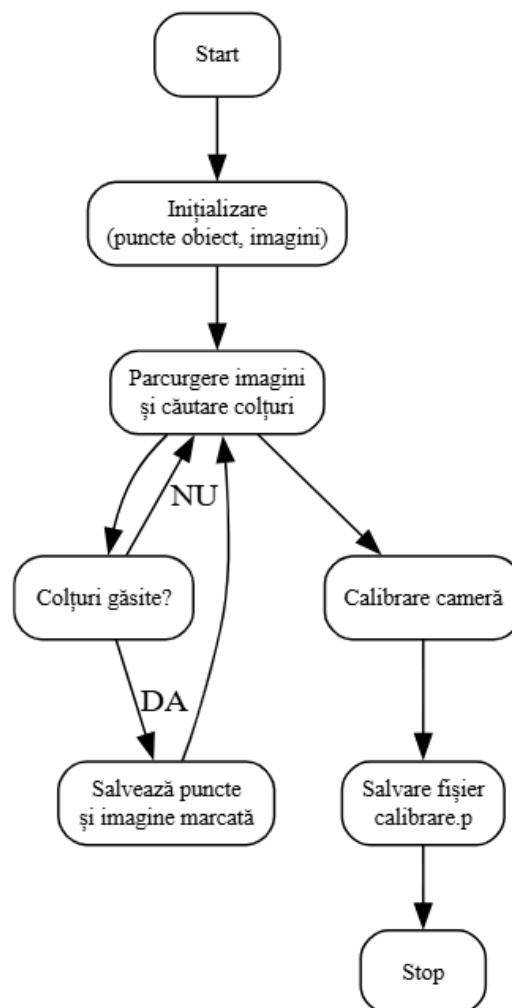


Figura 3.1.4 – Diagrama procesului de calibrare al camerei

Procesul de calibrare se bazează pe modelul geometric al camerei pinhole conform căruia un punct tridimensional $X = X \ Y \ Z$, este proiectat într-un punct bidimensional $x = x \ y$ din planul imaginii conform relației 1.

$$s \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = K \cdot [R \mid t] \cdot \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} \quad (1)$$

În această relație, K reprezintă matricea internă a camerei care conține parametrii intrinseci: precum distanțele focale și poziția centrului optic. Iar R și T definesc transformările extrinseci: adică rotația și translația camerei în raport cu sistemul de referință al lumii. Matricea K are forma prezentată în relația 2.

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \quad (2)$$

În implementarea practică, cu OpenCV, primul pas este definirea punctelor de control din spațiul real care în cazul acestui proiect sunt colțurile unei table de șah cu dimensiunile 9x6 puncte. Acestea sunt generate automat ca un set de coordonate 3D aflate în planul $Z = 0$ utilizând următoarea linie de cod:

```
objp[:, :2] = np.mgrid[0:9, 0:6].T.reshape(-1, 2)
```

Această instrucțiune construiește pozițiile teoretice ale colțurilor de pe tabla de șah în planul XY pentru fiecare imagine considerând că toate se află în același plan real.

Ulterior imaginile sunt convertite în tonuri de gri, iar colțurile sunt căutate prin metoda *findChessboardCorners* care returnează pozițiile acestora în planul imaginii dacă sunt detectate cu succes.

```
ret, corners = cv2.findChessboardCorners(gray, (9, 6), None)
```

Dacă rezultatul este pozitiv, atunci perechile de puncte obiect și puncte imagine sunt adăugate în seturile de date. Acestea vor fi transmise funcției de calibrare, astfel se construiește un set de corespondențe (X_i, x_i) pentru fiecare imagine. În final funcția *cv2.calibrateCamera* este utilizată pentru estimarea parametrilor camerei. Apoi aplică o metodă de optimizare ce minimizează eroarea de reproiecție. Adică diferența dintre colțurile detectate și cele proiectate matematic conform relației 1, linia de cod utilizată este:

```
ret, mtx, dist, rvecs, tvecs = cv2.calibrateCamera(objp_list, imgp_list, img_size, None, None)
```

Parametrii obținuți includ matricea intrinsecă K , vectorul coeficienților de distorsiune $dist = [k_1 k_2 p_1 p_2 k_3]$ și pentru fiecare imagine, matricea de rotație R și vectorul de translație t . Aceștia pot fi ulterior folosiți pentru corectarea imaginilor în timp real sau pentru proiectarea punctelor în spațiul real tridimensional.

Salvarea parametrilor într-un fișier binar, folosind linia de cod de mai jos, permite reutilizarea lor în alte scripturi fără a fi necesară recalibrarea completă de fiecare dată ceea ce este util mai ales în aplicații embedded sau autonome:

```
pickle.dump({'mtx': mtx, 'dist': dist}, open('calibrate.p', 'wb'))
```

3.2 IMPLEMENTAREA DETECȚIEI BENZILOR DE CIRCULAȚIE

În implementarea sistemelor de procesare a imaginilor destinate detectării benzilor de circulație, un rol esențial îl joacă utilizarea unor biblioteci specializate, care facilitează manipularea datelor vizuale și efectuarea operațiilor matematice complexe necesare. Astfel, în cadrul proiectului de față, au fost utilizate următoarele biblioteci:

NumPy, necesară pentru operații matriciale și prelucrări numerice rapide;

OpenCV, destinată procesării imaginilor și videoclipurilor, oferind funcționalități extinse precum conversii de culoare, aplicarea filtrelor sau corectarea distorsiunilor;

Pickle, utilizată pentru serializarea și încărcarea datelor salvate anterior, în special a parametrilor de calibrare a camerei.

Funcția ***undistort*** utilizează parametrii de calibrare salvați anterior (matricea camerei și vectorul de distorsiune) pentru a transforma fiecare imagine distorsionată într-o imagine rectificată, cum este ilustrat în figura 3.2.1 .

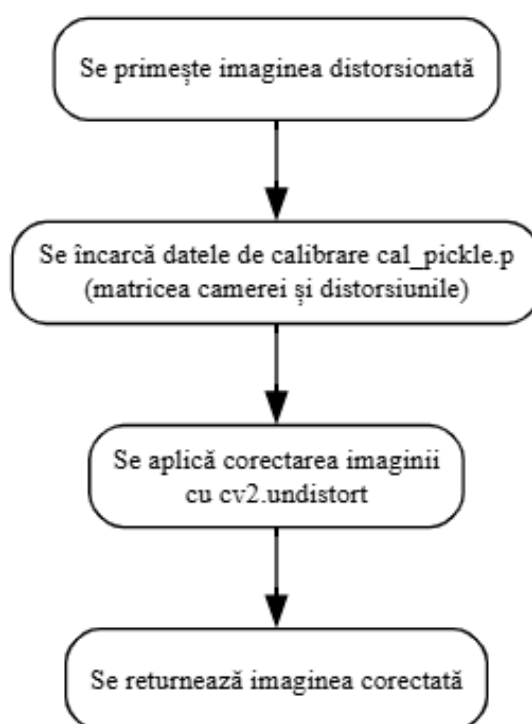


Figura 3.2.1 – Diagramă undistort

În figura 3.2.2 sunt ilustrate două exemple clare de aplicare a funcției ***undistort***, atât pe o tablă de șah utilizată pentru calibrare, cât și pe o imagine reală surprinsă din trafic.

În partea stângă se observă **imaginile originale**, unde liniile drepte par ușor curbate spre marginea imaginii, în special în colțuri. Acest efect este specific distorsiunii radiale negative (efect de tip „butoi”).

În partea dreaptă se află **imaginile corectate** folosind funcția ***undistort***. Se poate observa clar că pătratele tablei de șah devin perfect drepte și paralele, iar liniile de separare ale benzilor rutiere sunt net drepte și conforme cu realitatea.

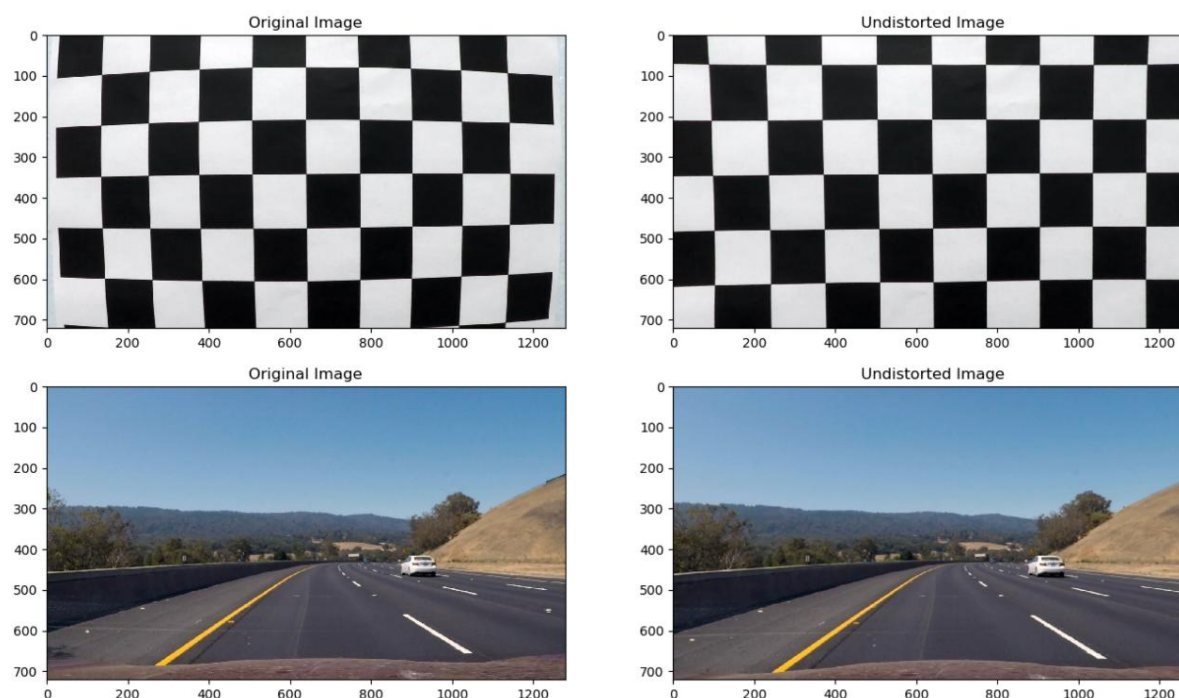


Figura 3.2.2 – Exemplu aplicare undistort

Această corectare permite algoritmilor de detecție să lucreze pe o geometrie fidelă, crescând semnificativ precizia în localizarea benzilor de circulație și a altor elemente de interes din imagine.

Funcția *pipeline* are rolul de a extrage o imagine binară dintr-o imagine color de intrare. Astfel punând în valoare regiunile, cu probabilitate ridicată, care ar putea conține benzile de circulație. Această extragere se realizează printr-un proces compus din conversii de culoare, detecția marginilor și filtrarea pe baza valorilor de saturație și gradient.

Fluxul de procesare începe prin conversia imaginii de intrare din spațiul de culoare RGB în spațiul HLS, pentru a separa componentele de culoare și luminozitate:

```
hls = cv2.cvtColor(img, cv2.COLOR_RGB2HLS)
```

Matematic, conversia se exprimă după relația 3. Unde $I(x, y)$ reprezintă imaginea inițială, iar $HLS(x, y)$ este imaginea în noul spațiu de culoare.

$$HLS(x, y) = f_{RGB \rightarrow HLS}(I(x, y)) \quad (3)$$

Separarea canalelor permite tratarea diferită a informației despre luminozitate și culoare, ceea ce ajută la robustețea detecției. Din imaginea HLS se extrag două canale, cel de luminozitate și cel de saturație:

```
I_channel = hls[:, :, 1]
s_channel = hls[:, :, 2]
```

Pe canalul de luminozitate, se aplică operatorul Sobel pentru calculul derivatelor parțiale de ordinul întâi în direcția orizontală x :

$$\text{sobelx} = \text{cv2.Sobel}(l_channel, \text{cv2.CV_64F}, 1, 0)$$

Această operație poate fi exprimată matematic prin relația 4, astfel:

$$G_x(x, y) = \frac{\partial L(x, y)}{\partial x} \quad (4)$$

având rolul de a evidenția tranzițiile bruște de intensitate orizontale, caracteristice marginilor verticale ale benzilor de circulație. Rezultatul gradientului este apoi normalizat într-un interval standardizat de valori $[0, 255]$, pentru a facilita pragul de selecție ulterioară:

$$\text{scaled_sobel} = \text{np.uint8}(255 * \text{np.absolute}(\text{sobelx}) / \text{np.max}(\text{np.absolute}(\text{sobelx})))$$

Normalizarea gradientului este exprimată prin relația 5. Această normalizare este necesară pentru a aduce toate valorile gradientului pe o scară comună, indiferent de variațiile absolute de luminozitate din imagine.

$$G'_x(x, y) = 255 \times \frac{|G_x(x, y)|}{\max(|G_x(x, y)|)} \quad (5)$$

Se creează apoi o imagine binară pentru gradientul Sobel, cum este ilustrat în figura 3.2.3. Această filtrare selectează pixelii care au valori ale gradientului între două praguri stabilite. În paralel, se procesează canalul de saturatie, creându-se o imagine binară, vezi figura 3.2.4, care selectează pixelii al căror nivel de saturație este cuprins între pragurile definite. Această selecție ajută la detectarea benzilor de circulație alb-galbene, indiferent de variațiile generale de culoare ale carosabilului.



Figura 3.2.3 – Aplicarea filtrului Sobel



Figura 3.2.4 – Aplicarea canalului de saturație

După obținerea celor două imagini binare, acestea sunt combinate, cum este ilustrat în figura 3.2.5 conform relației 6. Așadar, un pixel este setat la valoarea 1 în imaginea finală $B(x, y)$ dacă îndeplinește cel puțin una dintre cele două condiții: gradient puternic sau nivel adecvat de saturație.

$$B(x, y) = M_{Sobel}(x, y) \vee M_{sat}(x, y) \quad (6)$$

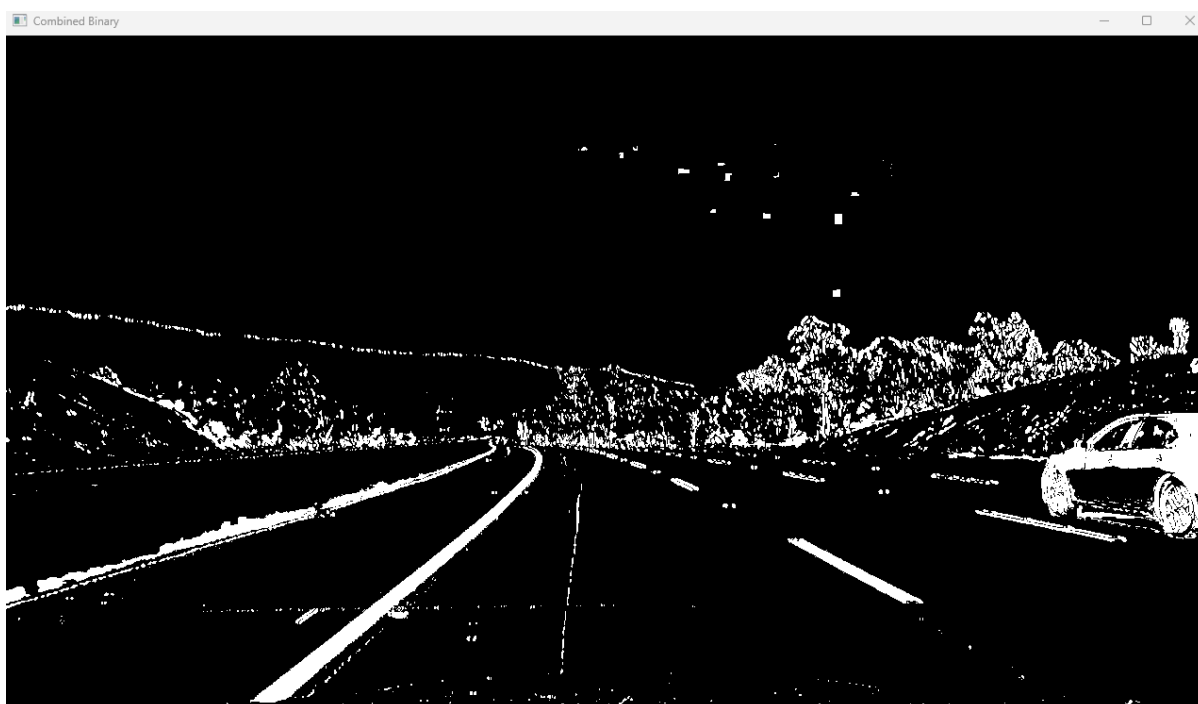


Figura 3.2.5 – Combinarea celor 2 filtre

Funcția *perspective_warp* transforma imaginea de intrare astfel încât drumul să pară privit dintr-o perspectivă verticală, adică de sus („bird's eye view”). Această transformare este vitală, pentru a face benzile de circulație să apară drepte și paralele, ceea ce simplifică extrem de mult procesul de detecție și urmărire a lor.

Primul pas constă în definirea dimensiunilor imaginii de lucru:

```
img_size = np.float32([(img.shape[1], img.shape[0])])
```

Această linie preia dimensiunile imaginii originale și le exprimă în tipul de date float32, necesar pentru funcțiile de transformare geometrică din OpenCV.

Următorul pas presupune scalarea coordonatelor sursă și destinație în funcție de dimensiunea imaginii:

```
src = src * img_size  
dst = dst * np.float32(dst_size)
```

Această scalare aduce coordonatele normalizate între 0 și 1 într-un sistem real de coordonate în pixeli. După ce punctele au fost definite, se calculează matricea de transformare proiectivă:

```
M = cv2.getPerspectiveTransform(src, dst)
```

Matematic, această matrice M este un tensor 3×3 care realizează o transformare de perspectivă (homografie) între planurile sursă și destinație. Aceasta satisface relația 7:

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = M \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \quad (7)$$

Unde (x', y') sunt noile coordonate proiectate și w' este factorul de scalare omogenă. După determinarea matricei M , imaginea este efectiv transformată prin aplicarea acestei matrice, conform figurii 3.2.6 .

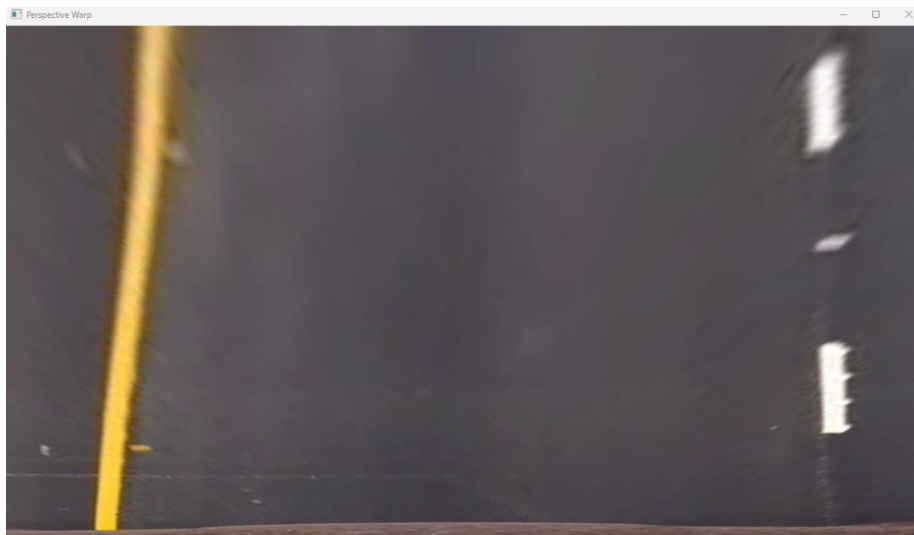


Figura 3.2.6 – Perspective Transform

Funcția *inv_perspective_warp* inversează efectul funcției *perspective_warp*. Aceasta permite proiectarea rezultatelor procesării din perspectiva verticală înapoi în perspectiva camerei reale, cum este ilustrat în figura 3.2.7.

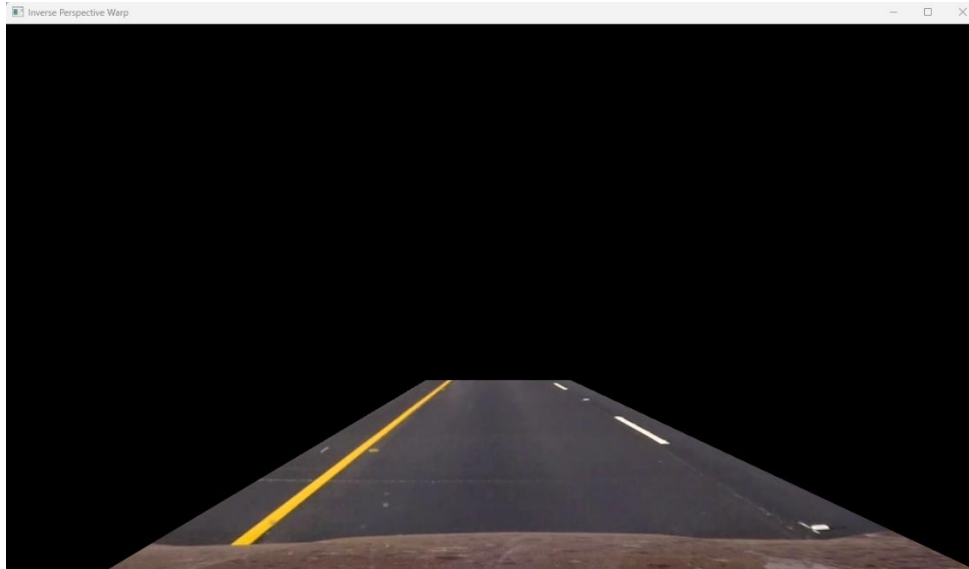


Figura 3.2.7 – Perspectiva reală a camerei

Funcția *sliding_window* are rolul de a detecta poziția benzilor de circulație într-o imagine binară rezultată în urma preprocesării. Această funcție funcționează prin utilizarea unui algoritm de scanare bazat pe ferestre glisante (*sliding windows*) care parcurge imaginea de jos în sus, căutând pixelii corespunzători benzilor de drum.

La început, se creează o imagine de ieșire care conține o copie a imaginii de intrare transformată într-un format color:

```
out_img = np.dstack((img, img, img)) * 255
```

Acest lucru este necesar pentru a putea vizualiza rezultatele detecției în culori diferite, evidențiind astfel benzile stângă și dreaptă.

Următorul pas constă în calcularea unei histogramme pe jumătatea inferioară a imaginii:

```
histogram = np.sum(img[img.shape[0] // 2:, :], axis=0)
```

Această histogramă $h(x)$ este definită matematic ca în relația 8:

$$h(x) = \sum_{y=\frac{H}{2}}^H B(x, y) \quad (8)$$

unde $B(x, y)$ este imaginea binară, H fiind înălțimea imaginii. Prin această sumare, obținem o distribuție orizontală a pixelilor activi, ceea ce permite estimarea poziției inițiale a benzilor. Se determină apoi punctele de pornire pentru benzile din stânga și din dreapta, folosind

valorile maxime ale histogramei. Astfel, se obține coordonata x corespunzătoare maximului de intensitate pentru banda stângă și pentru banda dreaptă.

Se determină înălțimea fiecărei ferestre glisante prin împărțirea înălțimii imaginii la numărul total de ferestre:

```
window_height = img.shape[0] // nwindows
```

Se identifică apoi coordonatele tuturor pixelilor activi (adică pixelii cu valoarea 1) din imagine. Inițial, pozițiile curente ale ferestrelor pentru stânga și dreapta sunt stabilite la bazele histogramei. Se creează două liste pentru a stoca indicii pixelilor găsiți pentru benzile stângă și dreaptă. Începe apoi procesul principal de scanare a imaginii de jos în sus, prin parcurgerea fiecărei ferestre glisante:

```
for window in range(nwindows):
```

Pentru fiecare fereastră se definesc limitele verticale și orizontale. Aceste limite definesc două dreptunghiuri (stânga și dreapta) care sunt folosite pentru a căuta pixelii activi din cadrul lor. Dacă opțiunea *draw_windows* este activată, se desenează aceste ferestre pe imaginea de ieșire. Dacă numărul de pixeli găsiți într-o fereastră este mai mare decât *minpix*, atunci poziția centrului ferestrei este actualizată ca fiind media pozițiilor pixelilor găsiți:

```
if len(good_left_inds) > minpix:  
leftx_current = int(np.mean(nonzerox[good_left_inds]))  
if len(good_right_inds) > minpix:  
rightx_current = int(np.mean(nonzerox[good_right_inds]))
```

Această re poziționare a ferestrelor permite urmărirea curbei benzii de circulație. După parcurgerea tuturor ferestrelor, listele de indici sunt concatenate:

```
left_lane_inds = np.concatenate(left_lane_inds)  
right_lane_inds = np.concatenate(right_lane_inds)
```

Se extrag coordonatele efective ale pixelilor corespunzători benzilor și se ajustează două polinoame de gradul 2 (parabolă) pentru benzile stânga și dreapta, folosind metoda celor mai mici pătrate:

```
left_fit = np.polyfit(lefty, leftx, 2)  
right_fit = np.polyfit(righty, rightx, 2)
```

Se calculează pozițiile corespunzătoare pentru fiecare curbă și în final, funcția returnează imaginea de ieșire colorată, cum este ilustrat în figura 3.2.8, coordonatele curbelor ajustate și coeficienții polinoamelor:

```
return out_img, (left_fitx, right_fitx), (left_fit, right_fit), ploty
```

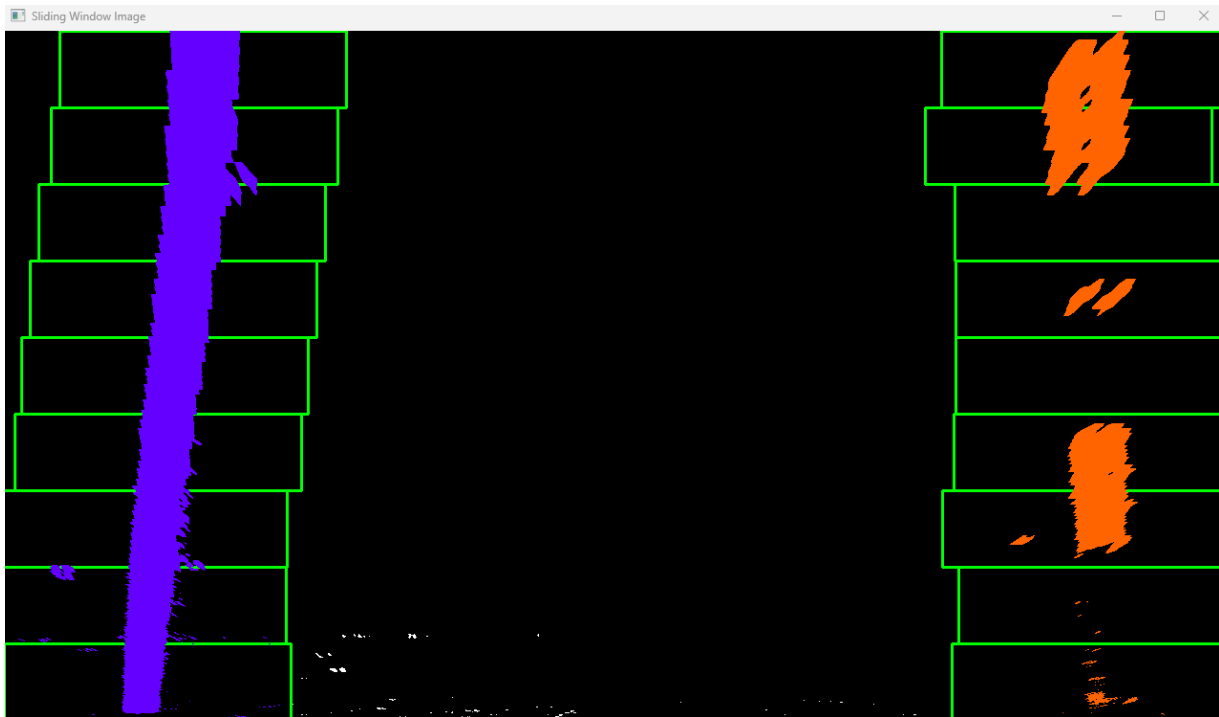


Figura 3.2.8 – Sliding window

Funcția *draw_lanes* are rolul de a desena benzile de circulație detectate pe imaginea originală, astfel încât traseul drumului să fie clar evidențiat.

Se începe prin generarea unui vector *ploty* care conține toate pozițiile verticale posibile (toate valorile posibile ale lui *y*) pe imaginea de ieșire:

```
ploty = np.linspace(0, img.shape[0] - 1, img.shape[0])
```

Acest vector reprezintă axa verticală, împărțită uniform între 0 și înălțimea imaginii minus 1.

Se creează apoi o imagine complet neagră cu aceeași dimensiune ca imaginea originală:

```
color_img = np.zeros_like(img)
```

Această imagine va fi utilizată pentru a desena zona dintre benzile detectate. Se construiesc coordonatele punctelor pentru banda stângă și banda dreaptă. Pentru banda stângă:

```
left = np.array([np.transpose(np.vstack([left_fit, ploty]))])
```

Se realizează o stivare (cu *vstack*) a coordonatelor (x_{left}, y), unde x_{left} este definit de polinomul ajustat anterior pentru banda stângă. Analog, pentru banda dreaptă, punctele benzii stânga și dreapta sunt concatenate pentru a forma o singură zonă închisă. Această zonă este apoi umplută cu o culoare verde.

Funcția *process_frame* reprezintă lanțul complet de procesare aplicat asupra unui cadru extras dintr-un flux video sau asupra unei imagini statice. Scopul principal al acestei funcții este de a detecta benzile de circulație și de a le reprezenta vizual suprapuse peste imaginea originală.

După definirea și implementarea funcțiilor dedicate prelucrării imaginilor, fluxul de procesare a fost aplicat iterativ pe fiecare cadru al unui videoclip de test. Utilizând metodele oferite de biblioteca OpenCV pentru manipularea fluxurilor video, s-a reușit prelucrarea în timp real a imaginilor.

Rezultatul final al procesării, prezentând benzile de circulație evidențiate, este redat în Figura 3.2.9.



Figura 3.2.9 – Rezultatele finale

4 SISTEM DE DETECȚIE A INDICATOARELOR RUTIERE

Proiectul de față se concentrează pe dezvoltarea unui sistem de detecție automată a indicatoarelor rutiere utilizând tehnici moderne de viziune artificială și inteligență artificială. Scopul principal este crearea unei aplicații capabile să analizeze în timp real imagini captate de o cameră video, identificând și clasificând corect semnele de circulație, contribuind astfel la realizarea unor sisteme avansate de asistență rutieră.

Pentru realizarea acestui sistem, s-au utilizat bibliotecile **OpenCV** și **TensorFlow Lite**. OpenCV a fost folosit pentru procesarea imaginii: conversia la grayscale, aplicarea de filtre de estompare Gaussiană, detectarea marginilor cu algoritmul Canny, extragerea conturilor și izolarea regiunilor de interes (ROI = region of interest) corespunzătoare indicatoarelor. Ulterior, regiunile extrase au fost clasificate folosind un model de rețea neuronală optimizat în format TensorFlow Lite (.tflite), rulând direct pe platforma Raspberry Pi pentru o execuție eficientă și rapidă.

Sistemul detectează conturile semnelor rutiere, extrage caracteristicile relevante și clasifică fiecare obiect într-o categorie predefinită (ex: STOP, YIELD, LIMITĂ DE VITEZĂ) cu un grad ridicat de acuratețe. Rezultatul este afișat atât pe imaginea procesată, cât și pe un ecran LCD1602 conectat la dispozitiv.

Prin implementarea acestui sistem, proiectul demonstrează cum tehnologiile moderne de procesare a imaginilor și rețelele neuronale compacte pot fi utilizate eficient pentru creșterea siguranței rutiere, oferind o soluție portabilă, rapidă și scalabilă pentru recunoașterea indicatoarelor în timp real.

4.1 INSTALAREA SISTEMULUI DE OPERARE PE RASPBERRY PI

Pentru a putea începe lucrul efectiv, în placa Raspberry Pi trebuie să fie introdus cardul de memorie microSD. Înainte de a face acest lucru însă trebuie să se pregătească sistemul de operare dacă acesta nu vine pre-instalat. Raspberry Pi poate rula diverse distribuții de sisteme de operare, precum Linux și o versiune minimală a sistemului de operare Microsoft Windows 10, respectiv Windows 10 IoT Core. În următorii pași se poate observa cum se instalează RaspberryPi OS, sistemul de operare oficial oferit de către Raspberry Pi a cărui grad de folosință este foarte ușor și e recomandat utilizatorilor noi în acest domeniu. Observație, pentru a monta imaginea sistemul de operare pe cardul microSD este necesar un adaptor de card, microSD-SD [7] [15] .

Pentru instalarea sistemului de operare pe cardul SD se urmărește următorul scenariu. Se descarcă Raspberry Pi Imager de pe adresa oficială a producătorului, <https://www.raspberrypi.org/downloads/raspbian/> . Raspberry Pi Imager este modalitatea rapidă și ușoară de a instala un sistem de operare pe un card microSD gata de utilizare cu Raspberry Pi. Acest program nu necesită instalare ci doar o simplă rulare a programului. În interfața grafică a programului se alege locația unde se va descărca sistemul de operare Raspbian, dispozitivul pe care dorim să se monteze imaginea, în cazul nostru cardul microSD iar apoi apăsați comanda "NEXT", vezi figura 4.1.1. După ce procesul s-a finalizat

cu succes puteți scoate cardul microSD din mașina personală și îl puteți introduce în placa Raspberry Pi [7] [15].

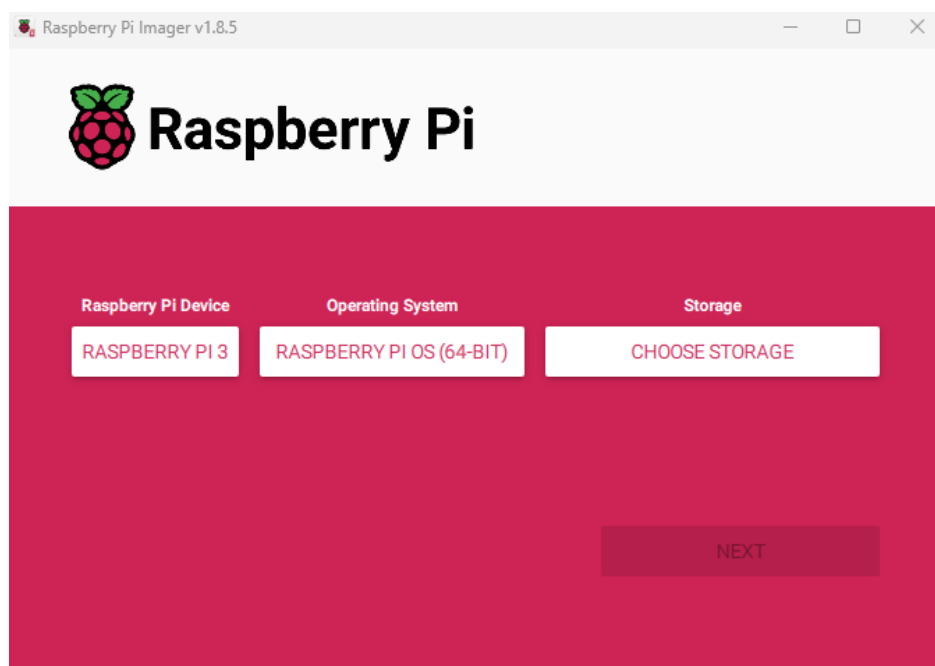


Figura 4.1.1 – Interfață Raspberry Pi Imager [15]

După ce s-a efectuat procesul de instalare a sistemului de operare se pot face anumite configurări. Aceste modificări pot fi făcute fie din interfața grafică sau din comenzi scrise în terminal.

Pentru a deschide terminalul, putem fie să accesăm pictograma sistemului de operare- >Accessories>Terminal sau apăsând pe tastatură combinația Ctrl+Alt+T iar pentru a accesa exact această fereastră se poate scrie în terminal, următoarea comandă "*sudo raspi-config*".

Comanda *sudo* permite execuția unor comenzi privilegiate, adică doar cu drepturi de administrare a sistemului

După instalarea sistemului de operare este recomandat procesul de schimbare a parolei predefinite de către sistemul de operare pentru a nu avea anumite neplăceri. Predefinit, utilizatorul are numele de *pi* iar parola este *raspberry*.

Dacă nu sunt folosite metode de a accesa placa Raspberry Pi de la distanță, serviciile SSH și VNC sunt recomandate a fi oprite.

De altfel, pentru a vă asigura că sistemul de operare are ultimele pachete instalate, este recomandată rularea a următoarelor comenzi în terminal:

```
sudo apt-get update  
sudo apt-get upgrade
```

4.2 CONFIGURAREA MEDIULUI DE DEZVOLTARE

Pentru a putea dezvolta și rula sistemul de detecție a indicatoarelor rutiere pe Raspberry Pi, a fost necesară configurarea atentă a mediului software.

Această etapă a presupus instalarea sistemului de operare, configurarea interfețelor hardware necesare și instalarea bibliotecilor de procesare a imaginilor și inteligență artificială.

4.2.1 Instalare OpenCV

Biblioteca OpenCV (Open Source Computer Vision Library) este utilizată pentru toate operațiile de procesare a imaginilor în cadrul proiectului cum ar fi conversia imaginilor din RGB în grayscale aplicarea de filtre de estompare detectarea marginilor și extragerea contururilor [5]. Instalarea bibliotecii OpenCV se realizează cu comanda:

```
sudo apt install python3-opencv
```

Această bibliotecă oferă suport extins pentru manipularea imaginilor și este indispensabilă pentru toate etapele de preprocesare vizuală implicate în detectarea indicatoarelor rutiere.

4.2.2 Instalare TensorFlow Lite Runtime

Pentru rularea modelului de clasificare TensorFlow Lite a fost necesară instalarea unei versiuni optimizate a bibliotecii TensorFlow denumită TensorFlow Lite Runtime care este destinată dispozitivelor cu resurse limitate precum Raspberry Pi [16]. Instalarea s-a efectuat folosind comanda:

```
pip install tf-lite-runtime
```

Această bibliotecă permite încărcarea modelelor de clasificare antrenate în prealabil în format .tflite și rularea predicțiilor într-un mod eficient din punct de vedere al resurselor

4.2.3 Instalare Picamera2

Pentru capturarea imaginilor în timp real s-a utilizat biblioteca Picamera2 care oferă suport nativ pentru noile camere compatibile cu Raspberry Pi și utilizează interfața Video for Linux V4L2 [4]. Instalarea Picamera2 s-a realizat folosind comanda:

```
pip install picamera2
```

De asemenea a fost necesară activarea camerei în configurația Raspberry Pi accesând utilitarul *sudo raspi-config* și selectând Interfacing Options urmat de Camera și Enable pentru activarea funcției de captură video.

4.2.4 Instalarea suportului pentru afișajul LCD1602

Pentru a afișa rezultatele clasificării în timp real a fost utilizat un ecran LCD1602 care comunică prin interfața I2C. Pentru a asigura comunicarea corectă au fost instalate pachetele:

```
pip install smbus2  
pip install RPLCD
```

În plus a fost necesară activarea interfeței I2C prin același utilitar *raspi-config* alegând opțiunea Interfacing Options și apoi I2C Enable. Aceste biblioteci permit transmiterea comenzilor către display prin magistrala serială I2C și afișarea în timp real a textelor reprezentând clasele detectate

4.3 Modelul de recunoaștere a indicatoarelor rutiere

Modelul utilizat în cadrul acestui proiect este un model pre-antrenat cu algoritmul YOLOv8, antrenat inițial pe baza de date *German Traffic Sign Recognition Benchmark (GTSRB)*. Acest set de date standardizat este utilizat frecvent în cercetarea și dezvoltarea sistemelor de recunoaștere automată a semnelor rutiere datorită complexității și diversității imaginilor pe care le conține [17].

Baza de date GTSRB oferă imagini reale cu indicatoare rutiere capturate în condiții variate de iluminare, unghiuri și distanțe, acoperind peste 40 de clase de semne rutiere. Această varietate asigură o pregătire robustă a modelului, oferindu-i capacitatea de a generaliza bine în condiții reale de trafic [17].

De când a fost introdus modelul YOLO, seria a suferit mai multe actualizări și iterații, cu performanțe îmbunătățite continuu. Fiind cea mai recentă dezvoltare din seria de modele YOLO, YOLOv8 reprezintă apogeul actual al tehnologiei. Arhitectura sa, descrisă în figura 4.3.1 cuprinde trei componente principale: coloana vertebrală, capul de detecție și gâtul.

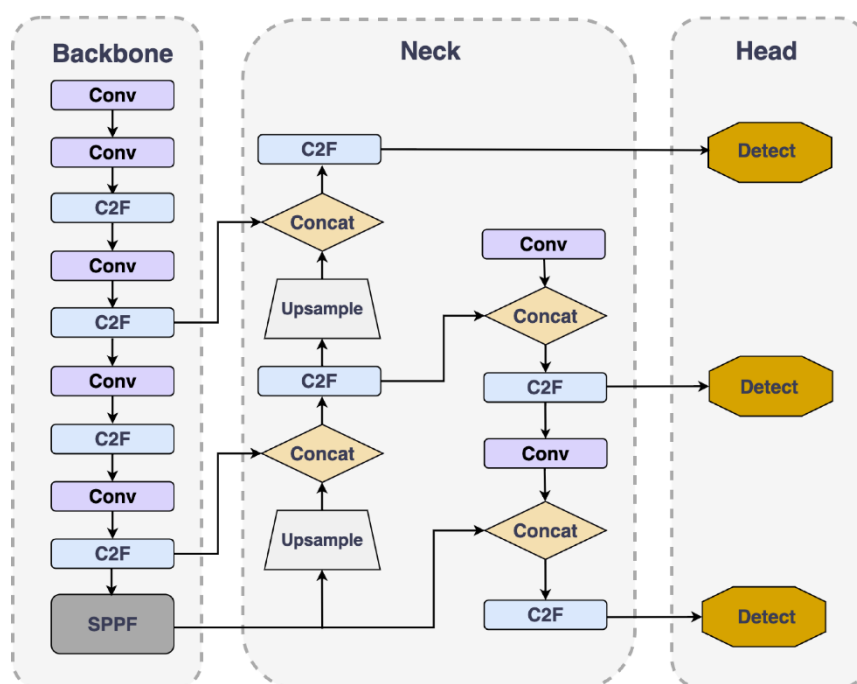


Figura 4.3.1 – Arhitectura yolov8 [18]

Backbone are rolul de a extrage caracteristici importante din imaginea inițială Conv sunt straturi de convoluție care aplică filtre pentru a scoate informații locale ca linii sau margini C2F este un modul nou introdus în YOLOv8 numit Cross Stage Partial Fusion care ajută la reducerea complexității și păstrarea performanței SPPF adică Spatial Pyramid Pooling Fast extrage informații de la diferite scale și face modelul mai robust la obiecte de mărimi diferite [18].

Neck combină caracteristicile extrase de Backbone la diferite rezoluții Concat concatenează adică lipește feature maps de la mai multe nivele Upsample mărește rezoluția imaginilor pentru a detecta obiecte mici iar Conv și C2F procesează mai departe caracteristicile combinate [18].

Head are rolul de a face efectiv predicțiile despre obiecte Sunt trei straturi Detect fiecare lucrând la o altă scară de imagine mică medie și mare pentru a putea găsi obiecte de diferite dimensiuni Fiecare strat Detect generează bounding boxes și clasificare.

În principiu Backbone extrage ce e important din imagine Neck combină și ajustează informațiile extrase iar Head detectează obiectele [18]

Conversia modelului YOLOv8 într-un format compatibil cu TensorFlow Lite s-a realizat utilizând biblioteca **Ultralytics YOLO**, printr-o comandă simplă în Python:

```
from ultralytics import YOLO
model = YOLO('yolov8n.pt')
model.export(format='tflite')
```

4.4 IMPLEMENTAREA DETECȚIEI INDICATOARELOR RUTIERE

Pentru implementarea sistemului de detecție și clasificare a indicatoarelor rutiere folosind un Raspberry Pi, a fost necesară importarea unor biblioteci specializate. Biblioteca NumPy a fost utilizată pentru operații matriciale rapide și eficiente, iar OpenCV a fost responsabilă pentru procesarea imaginilor capturate. De asemenea, modulul *Interpreter* din pachetul *tflite_runtime* a fost folosit pentru rularea modelului de rețea neuronală în format TensorFlow Lite, biblioteca LCD1602 pentru afișarea rezultatului pe un ecran LCD, iar Picamera2 pentru controlul camerei de captură.

După importarea bibliotecilor, au fost setați parametrii de bază ai aplicației. Rezoluția cadrelor video a fost aleasă la 640x480 pixeli, un compromis între claritatea detaliilor și timpul de procesare necesar. Pragul de acceptare pentru clasificare, notat cu T , a fost setat la valoarea 0.75, ceea ce impune ca o predicție să fie considerată validă doar dacă probabilitatea maximă a clasei prezise satisface condiția: $P_{max} \geq 0.75$, unde P_{max} este probabilitatea asociată clasei cu cel mai mare scor.

Fontul ales pentru desenarea textului pe imagine a fost `cv2.FONT_HERSHEY_SIMPLEX`, datorită lizibilității ridicate și compatibilității excelente cu mediul de procesare OpenCV.

Ulterior, s-a inițializat obiectul camerei prin instanțierea clasei *Picamera2*. Dimensiunea imaginii a fost setată conform parametrilor definiți, iar fluxul video a fost pornit utilizând metoda `start()`:

```
picam2 = Picamera2()
picam2.preview_configuration.main.size = (frameWidth, frameHeight)
picam2.preview_configuration.main.format = "RGB888"
picam2.configure("preview")
picam2.start()
```

În paralel, modelul neuronal de clasificare, salvat anterior în format TensorFlow Lite, a fost încărcat în memorie. Prin apelul funcției **`allocate_tensors()`** s-a realizat alocarea automată a tensorilor necesari pentru inferență, iar inputul și outputul modelului au fost configurate corespunzător:

```
model = Interpreter(model_path="/home/pi/model.tflite")
model.allocate_tensors()
```

Afișajul LCD a fost inițializat pentru a putea reda textul detectărilor în timp real, asigurând ștergerea oricărui conținut rezidual existent prin comanda:

```
lcd = LCD1602.LCD1602(16, 2)
lcd.clear()
```

În vederea pregătirii imaginilor pentru clasificare, a fost definit un flux de preprocesare compus din conversie la tonuri de gri, egalizare a histogrammei pentru amplificarea contrastului și normalizare a valorilor pixelilor:

Conversia la tonuri de gri: Imaginea color, reprezentată în spațiul BGR (Albastru, Verde, Roșu), este transformată într-o imagine monocromă (alb-negru). Această conversie are rolul de a reduce complexitatea datelor, păstrând doar informația de intensitate a luminii. Matematic, conversia se realizează pentru fiecare pixel folosind formula din relația 9:

$$Gray = 0.299 \times R + 0.587 \times G + 0.114 \times B \quad (9)$$

unde R , G și B sunt componentele de culoare ale pixelului. Rezultatul este o imagine cu un singur canal de intensitate, având valori între 0 și 255.

Egalizarea histogrammei: Pentru imaginea convertită la tonuri de gri, se aplică egalizarea histogrammei pentru a îmbunătăți contrastul. Egalizarea urmărește redistribuirea valorilor pixelilor astfel încât intensitățile să ocupe mai uniform întregul interval $[0, 255]$. Această operație face ca detaliile să devină mai vizibile în imagine.

Normalizarea valorilor pixelilor: După egalizare, valorile pixelilor sunt împărțite la 255 pentru a fi aduse în intervalul $[0, 1]$. Această etapă este esențială pentru rețelele neuronale, care funcționează optim atunci când datele de intrare sunt scalate. Matematic, normalizarea fiecărui pixel se realizează prin relația 10:

$$Pixel_{Normalizat} = \frac{Pixel_{Original}}{255} \quad (10)$$

În final, imaginea este convertită în format de tip *float32*, necesar pentru compatibilitatea cu modelele TensorFlow Lite utilizate în clasificare.

Prin acest flux de preprocesare, imaginile cu semne rutiere sunt standardizate și optim pregătite pentru a fi introduse în modelul de clasificare, figura 4.4.1 .



Figura 4.4.1 Preprocesarea imaginii

Funcția **getClassName** are rolul de a traduce un indice numeric într-un nume de clasă corespunzător unui semn rutier. Aceasta primește ca parametru un număr întreg (classNo) și returnează un șir de caractere (string) dintr-o listă prestabilită, care reprezintă denumirea semnului detectat. De exemplu:

Dacă classNo = 0, funcția returnează 'Viteza 20'

Dacă classNo = 14, funcția returnează 'STOP'

Dacă classNo = 27, funcția returnează 'Pietoni'

Funcția **detect_shape** are rolul de a identifica forma geometrică a unui obiect conturat într-o imagine. Aceasta primește ca parametru un contur, adică o listă de puncte care definesc marginea obiectului, și returnează o denumire textuală a formei: „triangle”, „rectangle” sau „circle”.

Întregul flux logic al acestei funcții este ilustrat în figura 4.4.2, unde sunt reprezentate etapele principale: primirea conturului, calculul perimetrului, aproximarea conturului, numărarea vârfurilor, luarea deciziei privind forma și returnarea rezultatului.

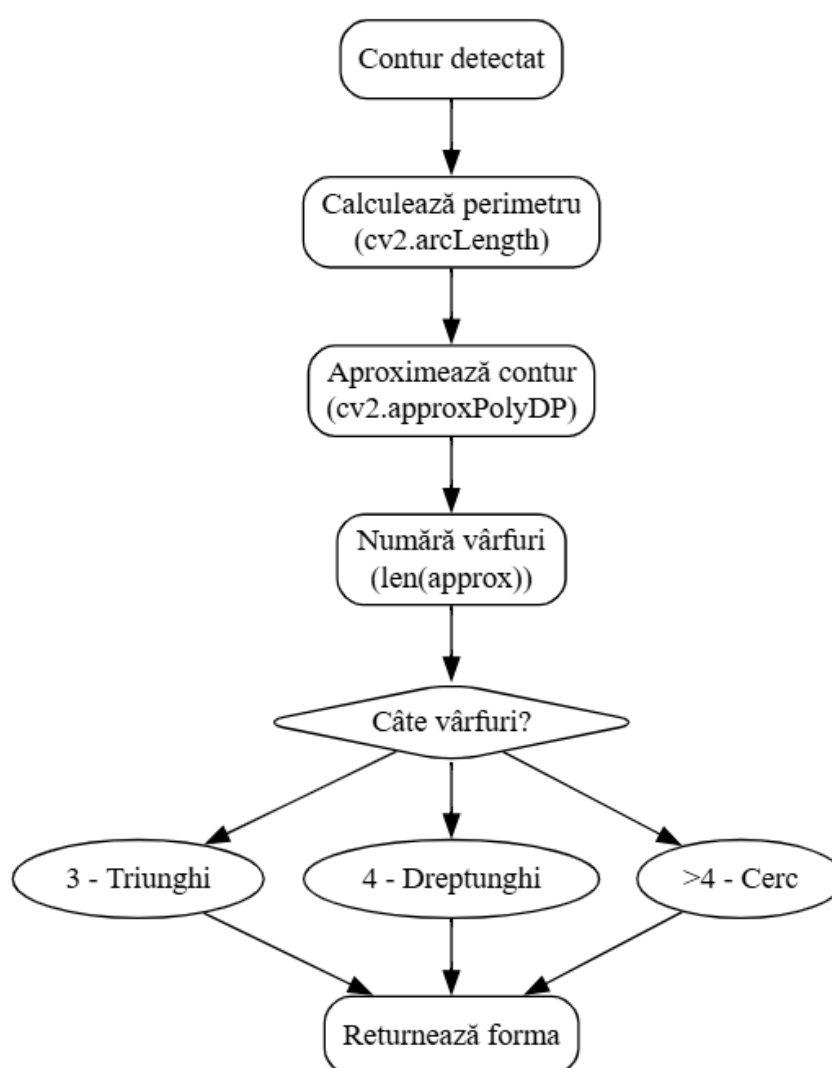


Figura 4.4.2 – Digrama detect_shape

În vederea pregătirii imaginii pentru detecția indicatoarelor, aceasta este supusă inițial unei operații de filtrare Gaussiană. Această prelucrare are rolul de a netezi imaginea și de a reduce zgomotul de înaltă frecvență, care ar putea produce detecții eronate ale marginilor. Operația este realizată prin funcția:

$$blur = cv2.GaussianBlur(gray, (5, 5), 1)$$

Funcția GaussianBlur aplică o convoluție între imagine și un nucleu Gaussian de dimensiune 5x5 pixeli, folosind o deviație standard (σ) de 1. Matematic, filtrul Gaussian este definit de funcția din relația 11:

$$G(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2+y^2}{2\sigma^2}\right) \quad (11)$$

Unde (x, y) sunt coordonatele pixelului față de centrul ferestrei și σ este deviația standard care controlează gradul de estompere.

Convoluția dintre nucleul Gaussian și imagine face ca fiecare pixel să fie înlocuit cu o medie ponderată a vecinilor săi, unde pixelii mai apropiați de centru au o influență mai mare. Ca efect, zgomotul (variațiile rapide de la un pixel la altul) este diminuat, dar marginile obiectelor rămân vizibile, cum este ilustrat în figura 4.4.3.

După această etapă de netezire, imaginea rezultată este procesată pentru detecția marginilor folosind algoritmul Canny:

$$edged = cv2.Canny(blur, 50, 150)$$

Algoritmul Canny este compus din mai multe etape succesive, fiecare având un rol bine definit:

Calculul gradientului de intensitate:

Se aplică operatorii Sobel pe direcțiile orizontală (X) și verticală (Y) pentru a estima variațiile de intensitate. Matematic, gradientul într-un pixel este ilustrat de relația 12:

$$G = \sqrt{G_x^2 + G_y^2} \quad (12)$$

Unde G_x și G_y sunt derivatele imaginii pe axele X și Y.

Suprimarea non-maximelor:

Se păstrează doar pixelii care sunt maxime locale ale gradientului, adică acele puncte care sunt cele mai intense în direcția gradientului. Această etapă subțiază marginile detectate.

Aplicarea pragurilor (thresholding dublu):

Se folosesc două praguri: un prag inferior (50) și un prag superior (150).

Urmărirea contururilor prin histerezis:

Pixelii de margine slabă sunt păstrați doar dacă sunt conectați la margini puternice, altfel sunt eliminați.

Se obține o imagine binară (edged) în care marginile obiectelor sunt bine evidențiate, fiind eliminate zonele zgomotoase sau variațiile ne semnificative, cum este ilustrat în figura 4.4.3.

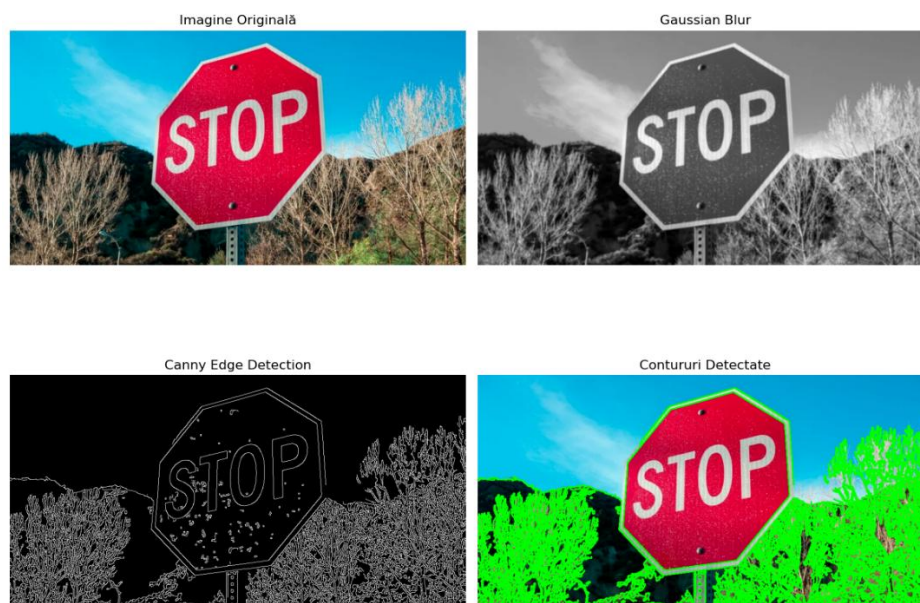


Figura 4.4.3 – Aplicare filtru Gaussian și Canny

Această metodă permite localizarea precisă a semnelor rutiere sau a altor forme geometrice într-un mod robust și eficient, esențial pentru funcționarea corectă a algoritmului de recunoaștere implementat.

După procesarea fiecărui contur valid detectat în imagine, zona de interes (ROI) este extrasă, vezi figura 4.4.4 și adaptată pentru a fi introdusă în rețeaua neuronală TensorFlow Lite.

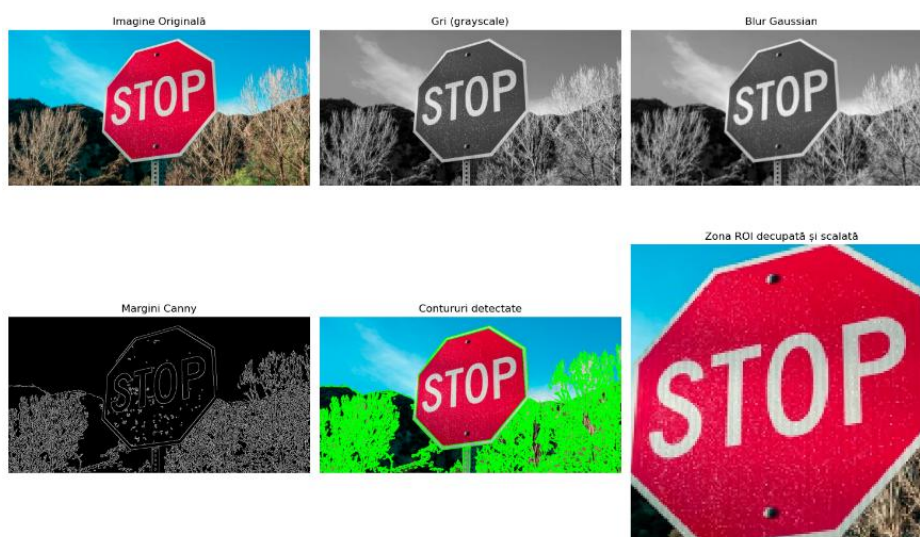


Figura 4.4.4 – Zona de interes extrasă

După extragerea și validarea conturilor din imagine, procesul de recunoaștere a semnelor rutiere avansează către etapa de clasificare automată, bazată pe rețele neuronale.

Fiecare zonă de interes (ROI) extrasă din cadrul original este supusă unui set de transformări standardizate: redimensionare la 32x32 pixeli, conversie la tonuri de gri, egalizare a histogramei și normalizare a valorilor pixelilor în intervalul [0, 1]. Aceste transformări au rolul de a aduce imaginea la un format compatibil cu cerințele modelului neuronal și de a asigura uniformitatea datelor de intrare, indiferent de condițiile de capturare (luminozitate, contrast etc.).

Zona astfel preprocesată devine **inputul pentru modelul TensorFlow Lite** încărcat anterior în memorie.

Modelul TensorFlow Lite utilizat este rezultatul unui proces separat de **antrenare**, realizat offline pe un set larg de imagini cu semne rutiere, unde rețeaua a învățat să diferențieze vizual între diverse clase, precum „STOP”, „Cedează trecerea”, „Viteză limitată” etc. Introducerea ROI-ului în rețea se realizează programatic astfel:

```
model.set_tensor(input_details[0]['index'], roi_processed)  
model.invoke()
```

Comanda *set_tensor* asociază imaginea procesată cu intrarea rețelei, iar *invoke* declanșează procesul de inferență, în care modelul analizează imaginea și generează predicțiile corespunzătoare.

Output-ul rețelei este un vector de probabilități, fiecare element reprezentând gradul de încredere al modelului că imaginea procesată aparține unei anumite clase. Predicția finală este determinată prin selecționarea clasei corespunzătoare probabilității maxime:

```
classIndex = np.argmax(predictions, axis=1)[0]  
probabilityValue = np.amax(predictions)
```

Pentru a filtra eventualele predicții nesigure, este implementat un prag de încredere (threshold = 0.75). Astfel, doar acele clasificări în care probabilitatea asociată depășește 75% sunt acceptate ca valide și utilizate mai departe.

În cazul unei clasificări reușite:

Numele clasei este obținut prin funcția *getClassName*.

Pe imaginea originală este desenat un chenar în jurul semnului detectat, împreună cu eticheta text care indică numele clasei și procentajul de încredere.

În paralel, informația este afișată și pe ecranul LCD1602 conectat la raspberry pi.

Această arhitectură permite un flux complet **automatizat** de recunoaștere a semnelor rutiere, de la capturarea imaginii, trecând prin procesare, clasificare inteligentă, și până la afișarea informațiilor pentru utilizator.

Figura 4.4.5 ilustrează arhitectura fluxului de procesare pentru recunoașterea automată a semnelor rutiere. Procesul începe cu **capturarea imaginii** de către camera sistemului, urmată de **detectia conturilor** pentru obținerea regiunilor potențial relevante. Ulterior, semnele suspectate sunt **decupate și procesate** (Region of Interest – ROI), fiind pregătite pentru etapa de **clasificare inteligentă**, prin intermediul unui model de rețea

neurală. În final, **informațiile sunt afișate** atât pe un LCD extern, cât și direct pe imaginea procesată, doar pentru verificare.

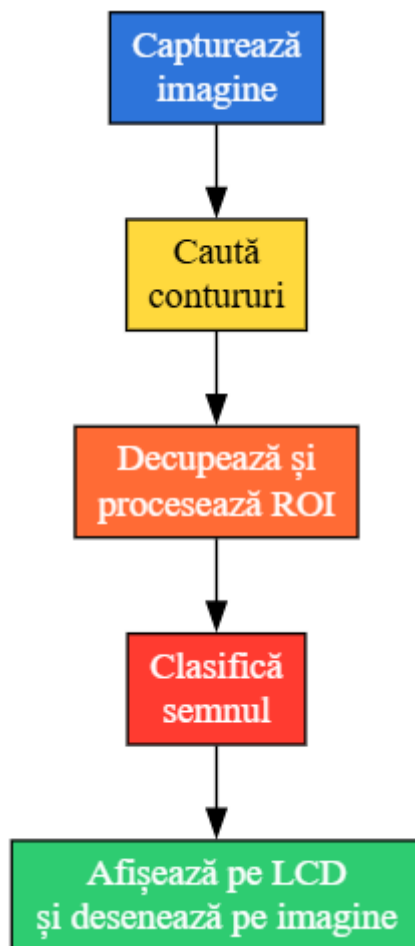


Figura 4.4.5 – Procesul de detecție simplificat

Astfel ansamblul permite un flux complet automatizat de recunoaștere a semnelor rutiere, de la capturarea imaginii, trecând prin procesare, clasificare inteligentă, și până la afișarea informațiilor pentru utilizator. Permițând astfel o funcționare rapidă, robustă și eficientă într-un sistem embedded.

4.5 REZULTATE EXPERIMENTALE

Montajul final, alcătuit din placa Raspberry Pi 3, camera oficială Raspberry Pi v2 și modulul de afișare LCD1602 conectat prin interfață I2C, a fost asamblat și montat pe un suportul portabil, de dimensiuni relativ mici. Întregul sistem este alimentat de un powerbank realizat cu două baterii Li-Ion tip 18650, care asigură mobilitatea și funcționarea autonomă a dispozitivului în condiții de testare în aer liber. Configurația completă a montajului este ilustrată în Figura 4.5.1.

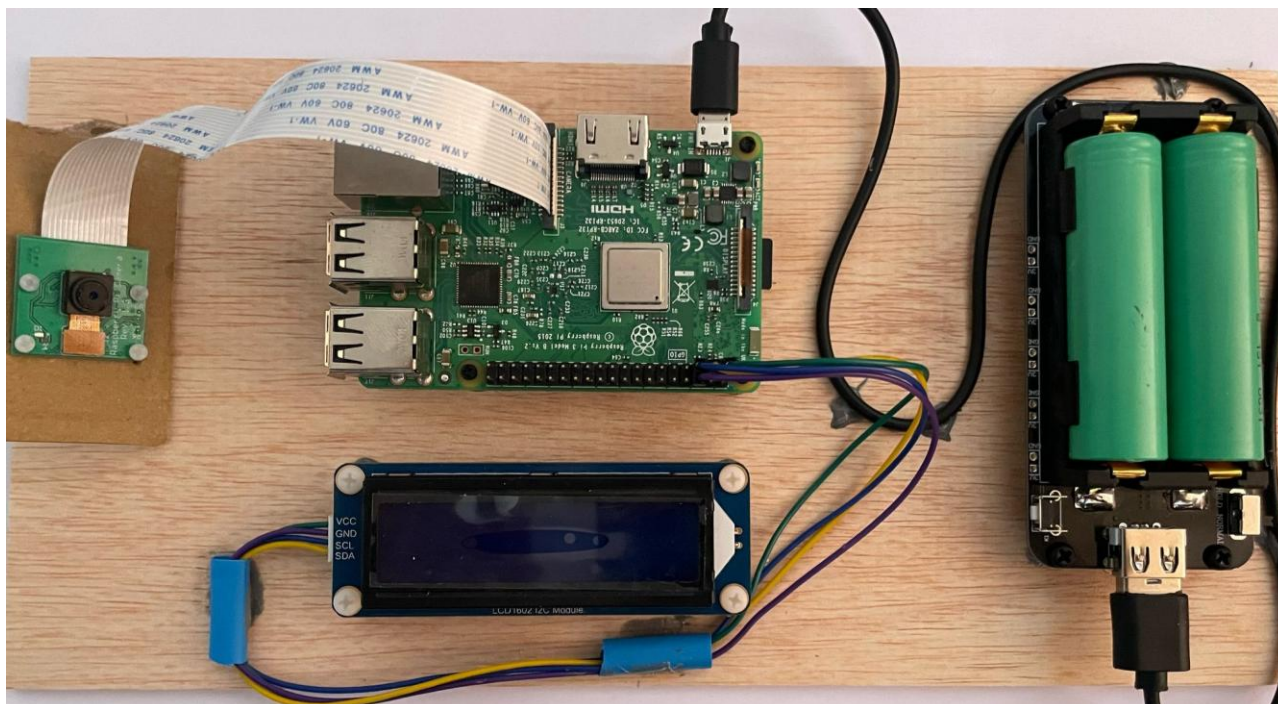


Figura 4.5.1 – Montaj complet

Pentru pornirea automată a aplicației, la lansarea sistemului, s-a utilizat mecanismul **crontab**. Mai întâi, a fost deschis terminalul, apoi s-a executat comanda:

`crontab -e`

În fișierul deschis, a fost adăugată linia:

`@reboot /home/pi/licenta.sh`

Această comandă asigură rularea scriptului `licenta.sh` la fiecare repornire a dispozitivului. Scriptul, aflat în directorul `/home/pi`, conține următoarele instrucțiuni, vizibile în figura 4.5.2.

```
~/licenta.sh - Mousepad
File Edit Search View Document Help
#!/bin/bash
cd /home/pi
python3 Final_Licenta.py
```

Figura 4.5.2 – Instrucțiunile pentru folderul `.sh`

Această implementare, are rolul de a lansa automat aplicația principală (`Final_Licenta.py`), oferind sistemului capacitate de funcționare autonomă imediat după boot, fără a fi nevoie de pornirea manuală a codului din terminal.

Pentru a facilita echilibru între viteză și precizie, a fost utilizată o rezoluție a imaginii de intrare de 320x240 pixeli, suficientă pentru detecția obiectelor aflate în apropiere, dar care limitează capacitatea de recunoaștere la distanțe mai mari. Din acest motiv, semnele

rutiere trebuie să fie plasate la o distanță mai mică de 30–40 cm față de cameră, astfel încât să ocupe o zonă suficient de mare în cadru pentru a putea fi detectate și clasificate corect.

Imaginea este analizată în timp real pentru identificarea conturilor, iar forma acestora este verificată pentru a filtra doar regiunile relevante care ar putea corespunde unor indicatoare rutiere. Regiunile valide sunt apoi redimensionate la 32x32 pixeli, normalizate și transmise către model, pentru clasificare. Afișarea rezultatului pe ecranul LCD1602 are loc doar dacă probabilitatea de clasificare depășește pragul de 75%, ceea ce contribuie la evitarea afișării unor rezultate eronate. Textul se actualizează numai atunci când este detectat un semn diferit față de cel anterior, iar în absența oricărei detecții timp de câteva secunde, afișajul este șters automat.

Acest comportament poate fi observat clar în figurile 4.5.3 – 4.5.6, unde se evidențiază momentul detecției, scorul de încredere și mesajul transmis pe ecran.

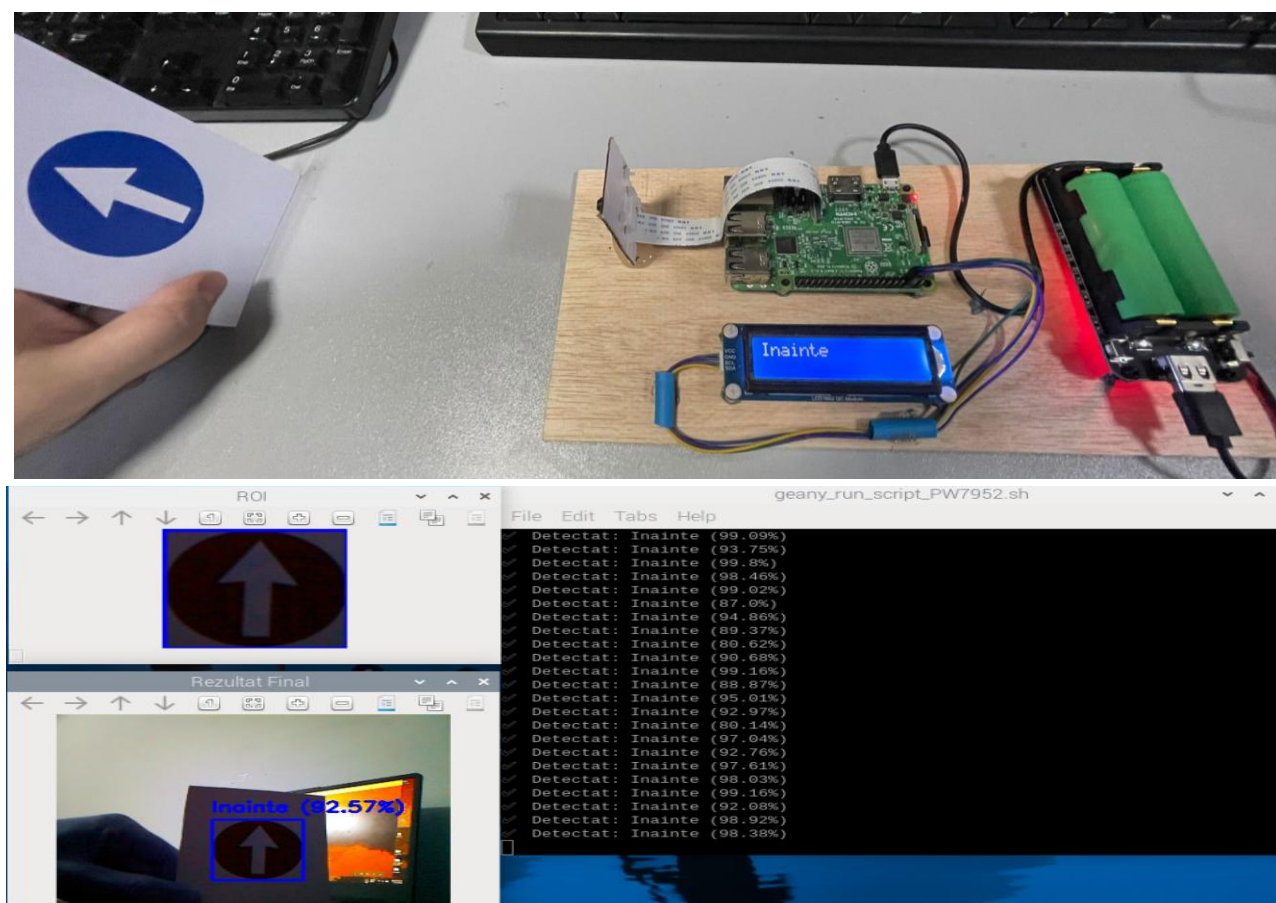


Figura 4.5.3 – Detecția semnului "Înainte"



Figura 4.5.4 – Detecția semnului "Dreapta Înainte"

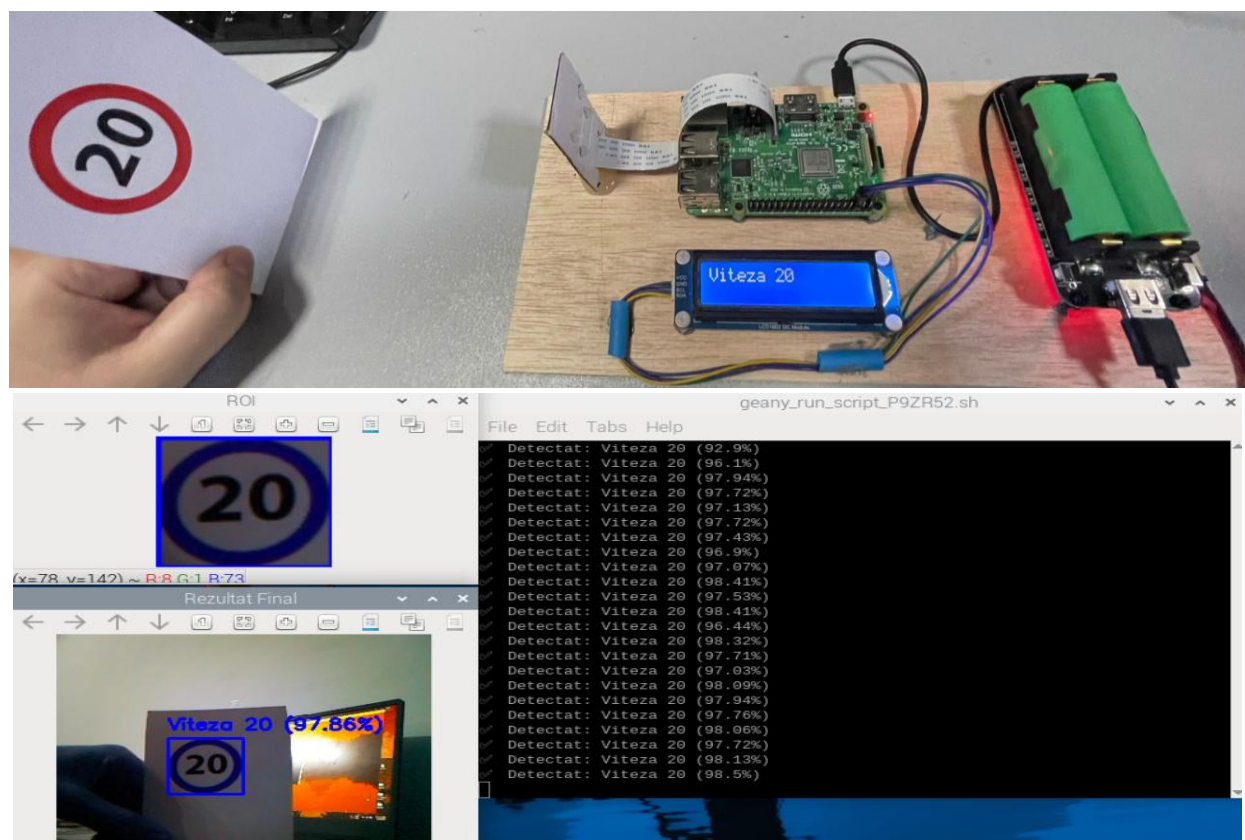


Figura 4.5.5 – Detecția indicatorului "Viteza maximă 20"

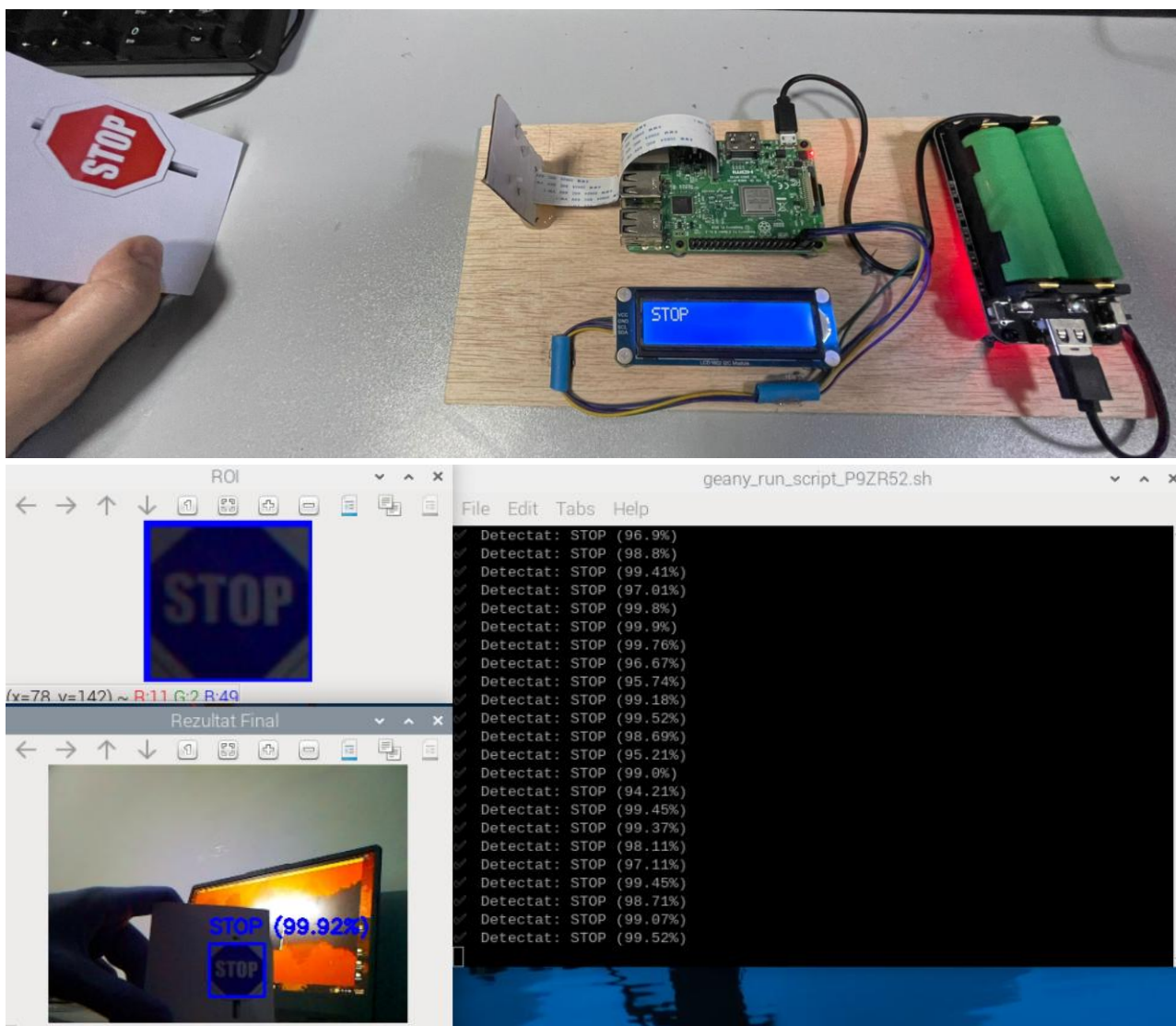


Figura 4.5.6 – Detecția indicatorului "Stop"

Sistemul funcționează doar în condiții de iluminare suficientă, deoarece calitatea imaginii este esențială pentru extragerea corectă a conturilor și pentru clasificare. Pe timp de noapte, sistemul poate funcționa cu ajutorul unei surse de lumină artificială, cum ar fi farurile unui autovehicul.

5 CONCLUZII

În documentul de față, a fost dezvoltat și implementat un sistem de viziune artificială destinat creșterii siguranței rutiere, prin detectarea benzilor de circulație și recunoașterea indicatoarelor rutiere. S-au utilizat tehnologii moderne de procesare a imaginilor și tehnici de clasificare bazate pe rețele neuronale, toate integrate într-un sistem embedded compact, bazat pe Raspberry Pi 3.

Componenta de detectare a benzilor de circulație s-a axat pe prelucrarea imaginilor prin conversii de culoare, aplicarea de filtre de estompare, detectarea marginilor cu algoritmul Canny și utilizarea metodei sliding window pentru identificarea benzilor. Aceste etape au fost esențiale pentru extragerea informațiilor relevante din imagini și pentru asigurarea unei funcționări robuste în condiții variate de trafic.

În paralel, componenta de recunoaștere a indicatoarelor rutiere a fost dezvoltată pe baza capturii live de imagini, procesării acestora și clasificării folosind modele optimizate TensorFlow Lite. Astfel, sistemul poate recunoaște semnele rutiere importante în timp real și poate afișa informațiile relevante pe un ecran LCD1602, oferind utilizatorului un feedback rapid și intuitiv.

Procesul de calibrare a camerei, realizat prin tehnici standardizate utilizând o tablă de șah, a avut un rol critic în corectarea distorsiunilor optice și în îmbunătățirea preciziei detecțiilor. Corelarea corectă a coordonatelor din lumea reală cu imaginile capturate a permis proiectarea unor algoritmi de detecție mai exacti și mai fiabili.

Rezultatele experimentale au demonstrat că sistemul este capabil să detecteze corect benzile de circulație și să recunoască indicatoarele rutiere într-un timp de procesare scăzut, compatibil cu cerințele aplicațiilor în timp real. De asemenea, utilizarea unei platforme cu resurse limitate, cum este Raspberry Pi 3, a evidențiat faptul că soluțiile eficiente de viziune artificială pot fi implementate și în medii embedded, fără a necesita echipamente costisitoare sau consumatoare de energie.

În ceea ce privește limitările identificate, precizia detectării benzilor poate fi afectată de condiții meteorologice dificile (ploaie abundentă, ceață) sau de imagini capturate în condiții de iluminare slabă. De asemenea, clasificarea indicatoarelor rutiere poate suferi în prezența unor semne degradate sau parțial acoperite.

Direcțiile viitoare de extindere a proiectului includ:

- Integrarea unor algoritmi de urmărire a vehiculelor și a pietonilor pentru un nivel mai ridicat de asistență rutieră.
- Utilizarea unor camere de rezoluție mai mare și a unor modele de rețea neuronală avansate pentru îmbunătățirea acurateței sistemului.

În concluzie, lucrarea demonstrează fezabilitatea implementării unor soluții accesibile și eficiente pentru asistență rutieră inteligentă, evidențiind potențialul real al tehnologiilor de viziune artificială integrate în sisteme embedded pentru creșterea siguranței în trafic.

BIBLIOGRAFIE

- [1] F. Ali, G. Ross, D. Santosh și R. Joseph, „You Only Look Once: Unified, Real-Time Object Detection,” *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, p. 779–788, 2016.
- [2] Defense Advanced Research Projects Agency, „Autonomous vehicle challenge led to new technologies and invigorated the prize challenge model of promoting innovation,” USA.gov, 13 Martie 2014. [Interactiv]. Available: <https://www.darpa.mil/news/2014/grand-challenge-ten-years-later>. [Accesat 10 Decembrie 2024].
- [3] Y. A. Ahmed, A. T. Mohamed și A. M. Bayoumy Aly, „Robust Lane Departure Warning System for ADAS on Highways,” în *2022 4th Novel Intelligent and Leading Emerging Sciences Conference (NILES)*, Giza, Egypt, 2022.
- [4] R. P. Foundation, „Raspberry Pi Documentation – Camera Module and AI Projects,” 14 Octombrie 2023. [Interactiv]. Available: <https://www.raspberrypi.com/documentation/accessories/camera.html>. [Accesat 14 Noiembrie 2024].
- [5] OpenCV, „OpenCV - Open Source Computer Vision Library,” [Interactiv]. Available: <https://opencv.org/>. [Accesat 7 Ianuarie 2025].
- [6] W. Pete și S. Daniel, *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*, Sebastopol: O'Reilly Media, 2019.
- [7] R. P. Foundation, „Getting started - Raspberry Pi Documentation,” 2025. [Interactiv]. Available: <https://www.raspberrypi.com/documentation/computers/getting-started.html>. [Accesat 3 Martie 2025].
- [8] R. P. Ltd, „Raspberry Pi 3 Model B+ Product Brief,” Ianuarie 2025. [Interactiv]. Available: <https://datasheets.raspberrypi.com/rpi3/raspberry-pi-3-b-plus-product-brief.pdf>. [Accesat 23 Februarie 2025].
- [9] R. P. Foundation, „Raspberry Pi Camera Module v2,” 2016.
- [10] Waveshare, „LCD1602 I2C Module,” 2 Aprilie 2021. [Interactiv]. Available: https://www.waveshare.com/wiki/LCD1602_I2C_Module. [Accesat 3 Ianuarie 2025].
- [11] diymore.cc, „18650 Battery Shield V8 Mobile Power Bank 3V/5V for Arduino ESP32 ESP8266 WiFi,” 7 Mai 2024. [Interactiv]. Available: <https://www.diymore.cc/products/18650-battery-shield-v8-mobile-power-bank-3v-5v-for-arduino-esp32-esp8266-wifi>. [Accesat 21 Aprilie 2025].

- [12] P. Institute, „Python® – the language of today and tomorrow,” [Interactiv]. Available: <https://pythoninstitute.org/about-python>. [Accesat 2 Octombrie 2024].
- [13] V. Tudor, Python 3 – Fundamente pentru începători, București: InfoBITS, 2019.
- [14] . J. Howse și J. Minichino, Learning OpenCV 4 Computer Vision with Python 3, Birmingham: Packt Publishing, 2019.
- [15] R. P. Foundation, „Raspberry Pi OS,” 2025. [Interactiv]. Available: <https://www.raspberrypi.com/software/>. [Accesat 3 Martie 2025].
- [16] G. AI, „Quickstart for Linux-based devices with Python,” 2025. [Interactiv]. Available: <https://ai.google.dev/edge/litert/microcontrollers/python>. [Accesat 3 Martie 2025].
- [17] R.-U. B. Institutul pentru Neuroinformatică, „German Traffic Sign Recognition Benchmark,” [Interactiv]. Available: <https://benchmark.ini.rub.de/>. [Accesat 4 Martie 2025].
- [18] Ultralytics, „A Guide on YOLOv8 Model Export to TFLite for Deployment,” [Interactiv]. Available: <https://docs.ultralytics.com/integrations/tflite/>. [Accesat 5 Martie 2025].